

Catálogo de Scripts — Análise de Algoritmos

20 scripts comentados, com perguntas prováveis e saída real · Prova com consulta de 22/09/2026

Índice de scripts

Índice de perguntas prováveis

Índice de palavras-chave

Scripts

- [01] Busca linear com contagem de passos
- [02] Busca binária iterativa com rastreamento de baixo/alto/meio
- [03] Busca binária recursiva (dividir para conquistar)
- [04] Benchmark: busca linear x busca binária (passos e tempo) — repete a Atividade 01
- [05] Selection sort com rastreamento de cada passagem
- [06] Bubble sort com parada antecipada e rastreamento de cada passagem
- [07] Crescimento das operações: o que acontece quando n dobra (bubble x selection)
- [08] Quicksort (versão do livro/aula) com rastreamento de pivô, partição e pilha
- [09] Quicksort: pivô primeiro x meio x aleatório (melhor, médio e pior caso) — experimento da aula 06
- [10] Merge sort com rastreamento das divisões e intercalações
- [11] Recursão básica: soma, contagem, máximo, fatorial e contagem regressiva (com profundidade da pilha)
- [12] Pilha de chamadas: o exemplo greet / greet2 do livro (cap. 3)
- [13] Fibonacci recursivo puro: árvore de chamadas, contagem e crescimento exponencial
- [14] Fibonacci com memorização (cache): dicionário manual e @lru_cache
- [15] Fibonacci: recursivo x memoizado x iterativo — tempo, chamadas e profundidade da pilha
- [16] Maior quadrado da fazenda (algoritmo de Euclides / MDC) — exemplo do livro e da aula 06
- [17] Reconhecendo o Big O de trechos de código: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$
- [18] Como medir tempo sem se enganar: perf_counter, timeit, mediana x média, seed e validação
- [19] Calculadora para a prova: passos da busca binária, $n(n-1)/2$, Fibonacci, chamadas, potências de 2
- [20] Caixeiro-viajante por força bruta: $O(n!)$ na prática (livro, cap. 1)

Como achar o que precisa. Use **Ctrl+F** neste arquivo (ou no PDF) com uma palavra-chave: o nome do algoritmo (bubble, quicksort, memo), um conceito (pior caso, pivô, caso base, n dobra) ou um trecho da pergunta. Os três índices abaixo apontam para o script certo. Cada script é um arquivo independente em scripts/ e roda sozinho com `python scripts/NN_nome.py`; a saída mostrada aqui é a saída real dessa execução.

No terminal, para procurar dentro dos scripts: `findstr /s /i "palavra" scripts\*.py` (Windows) ou `grep -ril palavra scripts/` (Git Bash).

Índice de scripts

#	Script	Tema	Complexidade	Palavras-chave
01	01_busca_linear.py Busca linear com contagem de passos	Busca	melhor $O(1)$, médio e pior $O(n)$	busca linear, busca sequencial, passos, pior caso, melhor caso, caso médio, alvo ausente, lista desordenada, for
02	02_busca_binaria.py Busca binária iterativa com rastreamento de baixo/alto/meio	Busca	melhor $O(1)$, médio e pior $O(\log n)$	busca binária, lista ordenada, meio, metade, log2, baixo, alto, chute, descartar metade, passos, 128, 1 milhão
03	03_busca_binaria_recursiva.py Busca binária recursiva (dividir para conquistar)	Busca / Dividir para conquistar	$O(\log n)$	busca binária recursiva, dividir para conquistar, caso base, caso recursivo, metade, profundidade da pilha, livro 4.4
04	04_benchmark_busca.py Benchmark: busca linear x busca binária (passos e tempo) — repete a Atividade 01	Busca / Medição	$O(n) \times O(\log n)$	benchmark, tempo, perf_counter, mediana, passos x tempo, busca linear, busca binária, pior caso, 1 milhão, atividade 01, speedup
05	05_selection_sort.py Selection sort com rastreamento de cada passagem	Ordenação	melhor, médio e pior $O(n^2)$	selection sort, ordenação por seleção, menor elemento, $n(n-1)/2$, comparações, trocas, sempre quadrático, livro cap. 2
06	06_bubble_sort.py Bubble sort com parada antecipada e rastreamento de cada passagem	Ordenação	melhor $O(n)$, médio e pior $O(n^2)$	bubble sort, ordenação por bolha, vizinhos, troca, passagem, houve_troca, parada antecipada, melhor caso $O(n)$, $n(n-1)/2$, atividade 02
07	07_crescimento_ordenacao.py Crescimento das operações: o que acontece quando n dobra (bubble x selection)	Ordenação / Big O	$O(n^2)$	n dobra, quadruplica, fator 4, $n(n-1)/2$, comparações, trocas, crescimento, tabela, random.seed, atividade 02, quadrático
08	08_quicksort.py Quicksort (versão do livro/aula) com rastreamento de pivô, partição e pilha	Dividir para conquistar / Ordenação	melhor e médio $O(n \log n)$, pior $O(n^2)$	quicksort, pivô, particionamento, menores, maiores, caso base, recursão, altura da pilha, chamadas, dividir para conquistar, aula 06
09	09_quicksort_pivos.py Quicksort: pivô primeiro x meio x aleatório (melhor, médio e pior caso) — experimento da aula 06	Dividir para conquistar / Ordenação / Medição	$O(n \log n)$ médio, $O(n^2)$ pior	quicksort, escolha do pivô, pivô aleatório, pivô do meio, random.choice, pior caso, lista ordenada, caso médio, tempo, comparações, altura da pilha, setrecursionlimit
10	10_merge_sort.py Merge sort com rastreamento das divisões e intercalações	Dividir para conquistar / Ordenação	melhor, médio e pior $O(n \log n)$	merge sort, intercalação, merge, dividir ao meio, combinar, $O(n \log n)$ garantido, estável, memória extra, altura log n
11	11_recurcao_basica.py Recursão básica: soma, contagem, máximo, fatorial e contagem regressiva (com profundidade da pilha)	Recursividade	$O(n)$ chamadas	recursão, caso base, caso recursivo, soma recursiva, contar, máximo, fatorial, regressiva, profundidade da pilha, RecursionError, stack overflow, livro 4.1 4.2 4.3
12	12_pilha_de_chamadas.py Pilha de chamadas: o exemplo greet / greet2 do livro (cap. 3)	Recursividade / Pilha	$O(1)$	pilha de chamadas, call stack, push, pop, greet, greet2, bye, chamada incompleta, suspensão, LIFO, livro 3.1

#	Script	Tema	Complexidade	Palavras-chave
13	13_fibonacci_recursivo.py Fibonacci recursivo puro: árvore de chamadas, contagem e crescimento exponencial	Recursividade / Fibonacci	tempo $O(2^n)$ (exatamente $\Theta(1,618^n)$)	fibonacci, recursivo puro, árvore de chamadas, subproblemas sobrepostos, recálculo, exponencial, razão áurea, phi 1.618, $2F(n+1)-1$, chamadas, atividade 03
14	14_fibonacci_memo.py Fibonacci com memorização (cache): dicionário manual e @lru_cache	Recursividade / Fibonacci / Cache	tempo $O(n)$	memorização, memoization, cache, dicionário, lru_cache, cache_info, hits, misses, $2n-1$, top-down, cache quente, RecursionError, atividade 03
15	15_fibonacci_comparacao.py Fibonacci: recursivo x memoizado x iterativo — tempo, chamadas e profundidade da pilha	Recursividade / Fibonacci / Medição	$O(2^n) \times O(n) \times O(n)$	fibonacci iterativo, bottom-up, comparação, benchmark, timeit, mínimo, tempo, memória, profundidade da pilha, sizeof, memória auxiliar, atividade 03
16	16_euclides_maior_quadrado.py Maior quadrado da fazenda (algoritmo de Euclides / MDC) — exemplo do livro e da aula 06	Dividir para conquistar / Recursividade	$O(\log(\min(a, b)))$	fazenda, 1680 x 640, maior quadrado, Euclides, MDC, resto da divisão, operador módulo, caso base múltiplo, dividir para conquistar, math.gcd, quadrado de 80
17	17_big_o_trechos.py Reconhecendo o Big O de trechos de código: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$	Big O	uma função para cada classe	big o, classes de complexidade, contar operações, laço duplo, dividir por 2, n dobra, fator, constante, logarítmico, linear, linearítmico, quadrático, exponencial, subconjuntos, identificar complexidade
18	18_como_medir.py Como medir tempo sem se enganar: perf_counter, timeit, mediana x média, seed e validação	Medição / Benchmark	—	perf_counter, time.time, timeit, repeat, mediana, média, mínimo, pico, ruído, random.seed, reprodutibilidade, assert, validar corretude, benchmark honesto, statistics
19	19_calculadora_prova.py Calculadora para a prova: passos da busca binária, $n(n-1)/2$, Fibonacci, chamadas, potências de 2	Ferramenta / Big O	—	calculadora, log2, potências de 2, ceil, floor, $n(n-1)/2$, fibonacci, $2F(n+1)-1$, $2n-1$, fatorial, tabela, estimar tempo, quanto tempo leva
20	20_caixeiro_viajante.py Caixeiro-viajante por força bruta: $O(n!)$ na prática (livro, cap. 1)	Big O / Força bruta	$O(n!)$	caixeiro viajante, travelling salesman, permutações, força bruta, fatorial, $n!$, itertools.permutations, inviável, explode, rotas

Índice de perguntas prováveis

Pergunta	Script
Qual é a complexidade da busca linear no pior caso e por quê?	01
A busca linear exige lista ordenada?	01
Quantos passos no melhor caso? E no caso médio?	01
Se a lista dobrar de tamanho, o que acontece com o pior caso?	01
Por que a lista precisa estar ordenada?	02
Quantos passos no máximo para $n = 128$? E para $n = 1.000.000$? (Livro, ex. 1.1)	02
O que acontece quando o alvo não está na lista?	02
Qual é o melhor caso da busca binária?	02
Se a lista dobra de tamanho, quantos passos a mais? (Livro, ex. 1.2)	02
Qual é o caso base e o caso recursivo da busca binária? (Livro, ex. 4.4)	03
Por que a busca binária é um exemplo de dividir para conquistar?	03
Qual é a profundidade máxima da pilha nessa versão?	03
A versão recursiva é mais rápida que a iterativa?	03
Por que medir passos E tempo?	04
Por que a razão de tempo é menor que a razão de passos?	04
Por que usar mediana em vez de média?	04
Por que usar uma lista grande?	04
Como funciona o selection sort?	05
Por que ele é $O(n^2)$ mesmo no melhor caso?	05
Quantas trocas ele faz no máximo?	05
Por que $O(n^2)$ e não $O(n^2/2)$? (Livro, cap. 2)	05
Como funciona o bubble sort?	06
Por que o melhor caso é $O(n)$?	06
Quantas comparações e trocas no pior caso (lista invertida)?	06
Por que o laço interno vai até $n - 1$ - passagem?	06
Bubble sort ou selection sort para uma lista quase ordenada?	06
Quando n dobra, quantas vezes crescem as comparações de um algoritmo $O(n^2)$?	07
Por que o melhor caso do bubble sort cresce só 2x quando n dobra?	07
Por que o selection sort faz o mesmo número de comparações nos três cenários?	07
Para que serve <code>random.seed(42)</code> aqui?	07
Qual é o caso base do quicksort?	08
Quais são as três etapas do quicksort?	08
Por que a lista ordenada é o pior caso com pivô no primeiro elemento?	08
O que muda entre melhor e pior caso: o custo de cada nível ou a quantidade de níveis?	08
Quantas chamadas de quicksort ocorrem?	08
Como evitar o pior caso do quicksort?	09

Pergunta	Script
Por que a aula usou <code>sys.setrecursionlimit</code> ?	09
Com pivô aleatório, qual é o Big O esperado?	09
Merge sort é sempre melhor por garantir $O(n \log n)$?	09
Quais são as etapas dividir, conquistar e combinar no merge sort?	10
Por que o merge sort é $O(n \log n)$ em todos os casos?	10
Qual é a desvantagem do merge sort em relação ao quicksort?	10
Quando preferir merge sort?	10
O que são caso base e caso recursivo?	11
O que acontece se esquecer o caso base? (Livro, ex. 3.2)	11
Qual é a profundidade da pilha em <code>soma([2, 4, 6])</code> ?	11
Em que ordem o fatorial multiplica?	11
O que a pilha de chamadas guarda?	12
Olhando a pilha <code>[greet2 name: maggie]</code> sobre <code>[greet name: maggie]</code> , o que se conclui? (Livro, ex. 3.1)	12
Qual é a relação entre pilha de chamadas e recursão?	12
Por que o Fibonacci recursivo puro é ineficiente?	13
Qual é a complexidade e como se chega a ela?	13
Se <code>fib(30)</code> leva 0,13 s, quanto leva <code>fib(40)</code> ?	13
O tempo é exponencial; a memória também?	13
O que é memorização e por que ela ajuda?	14
Quantas chamadas faz a versão memoizada?	14
Qual é o custo em memória da memorização?	14
Por que <code>fib_memo(5000)</code> pode dar <code>RecursionError</code> e a iterativa não?	14
O que é um cache quente?	14
Qual versão é a mais eficiente e por quê?	15
Por que validar a corretude antes do benchmark?	15
Por que usar o menor tempo (ou a mediana) e não a média?	15
Por que o cache é zerado a cada execução do benchmark?	15
Qual é o caso base do problema da fazenda?	16
Qual é o caso recursivo?	16
Mostre as reduções para 1680×640 .	16
Que algoritmo clássico é esse?	16
Por que a complexidade é logarítmica?	16
Como identificar o Big O de um trecho de código?	17
Qual é o fator de crescimento de cada classe quando n dobra?	17
<code>for i in range(n): for j in range(i+1, n): ...</code> é $O(n^2)$?	17
Um laço que faz <code>i = i // 2</code> até <code>i</code> chegar a 1 é $O(?)$	17
Por que repetir a medição e usar mediana ou mínimo?	18
Para que serve <code>random.seed(42)</code> ?	18

Pergunta	Script
Por que validar a corretude antes do benchmark?	18
Qual a diferença entre contar passos e medir tempo?	18
Quantos passos a busca binária faz para $n = 240.000$? E para 4 bilhões?	19
Quantas comparações faz o selection sort para $n = 1.000$?	19
Um algoritmo $O(n^2)$ leva 2 s para $n = 1.000$; quanto leva para $n = 10.000$?	19
Quantas chamadas faz fib_recurso(25)? E fib_memo(25)?	19
Por que o caixeiro-viajante por força bruta é $O(n!)$?	20
O que acontece quando se acrescenta uma cidade?	20
Qual é o algoritmo mais lento da tabela do livro?	20

Índice de palavras-chave

Procure a palavra (Ctrl+F) e vá ao script indicado.

1 milhão 02, 04	dicionário 14	melhor caso 01
128 02	dividir ao meio 10	melhor caso $O(n)$ 06
1680 x 640 16	dividir para conquistar 03, 08, 16	memoization 14
$2f(n+1)-1$ 13, 19	dividir por 2 17	memória 15
$2n-1$ 14, 19	escolha do pivô 09	memória auxiliar 15
alto 02	estável 10	memória extra 10
altura da pilha 08, 09	estimar tempo 19	memorização 14
altura log n 10	euclides 16	menor elemento 05
alvo ausente 01	explode 20	menores 08
árvore de chamadas 13	exponencial 13, 17	merge 10
assert 18	fator 17	merge sort 10
atividade 01 04	fator 4 07	metade 02, 03
atividade 02 06, 07	fatorial 11, 19, 20	mínimo 15, 18
atividade 03 13, 14, 15	fazenda 16	misses 14
aula 06 08	fibonacci 13, 19	n dobra 07, 17
baixo 02	fibonacci iterativo 15	n! 20
benchmark 04, 15	floor 19	$n(n-1)/2$ 05, 06, 07, 19
benchmark honesto 18	for 01	$O(n \log n)$ garantido 10
big o 17	força bruta 20	operador módulo 16
bottom-up 15	getsizeof 15	ordenação por bolha 06
bubble sort 06	greet 12	ordenação por seleção 05
busca binária 02, 04	greet2 12	parada antecipada 06
busca binária recursiva 03	hits 14	particionamento 08
busca linear 01, 04	houve_troca 06	passagem 06
busca sequencial 01	identificar complexidade 17	passos 01, 02
bye 12	intercalação 10	passos x tempo 04
cache 14	inviável 20	perf_counter 04, 18
cache quente 14	itertools.permutations 20	permutações 20
cache_info 14	laço duplo 17	phi 1.618 13
caixeiro viajante 20	lifo 12	pico 18
calculadora 19	linear 17	pilha de chamadas 12
call stack 12	linearítico 17	pior caso 01, 04, 09
caso base 03, 08, 11	lista desordenada 01	pivô 08
caso base múltiplo 16	lista ordenada 02, 09	pivô aleatório 09
caso médio 01, 09	livro 3.1 12	pivô do meio 09
caso recursivo 03, 11	livro 4.1 4.2 4.3 11	pop 12
ceil 19	livro 4.4 03	potências de 2 19
chamada incompleta 12	livro cap. 2 05	profundidade da pilha 03, 11, 15
chamadas 08, 13	log2 02, 19	push 12
chute 02	logarítmico 17	quadrado de 80 16
classes de complexidade 17	lru_cache 14	quadrático 07, 17
combinar 10	maior quadrado 16	quadruplica 07
comparação 15	maiores 08	quanto tempo leva 19
comparações 05, 07, 09	math.gcd 16	quicksort 08, 09
constante 17	máximo 11	random.choice 09
contar 11	mdc 16	random.seed 07, 18
contar operações 17	média 18	razão áurea 13
crescimento 07	mediana 04, 18	recálculo 13
descartar metade 02	meio 02	recursão 08, 11

recursionerror 11, 14
recursivo puro 13
regressiva 11
repeat 18
reprodutibilidade 18
resto da divisão 16
rotas 20
ruído 18
selection sort 05
sempre quadrático 05

setrecursionlimit 09
soma recursiva 11
speedup 04
stack overflow 11
statistics 18
subconjuntos 17
subproblemas sobrepostos 13
suspensa 12
tabela 07, 19
tempo 04, 09, 15

time.time 18
timeit 15, 18
top-down 14
travelling salesman 20
troca 06
trocas 05, 07
validar corretude 18
vizinhos 06

Scripts

[01] Busca linear com contagem de passos

Arquivo: scripts/01_busca_linear.py · **Tema:** Busca · **Complexidade:** melhor $O(1)$, médio e pior $O(n)$ · **Memória:** $O(1)$

Palavras-chave: busca linear, busca sequencial, passos, pior caso, melhor caso, caso médio, alvo ausente, lista desordenada, for

O que o script faz

- Percorre a lista item a item até achar o alvo, contando quantos itens olhou.
- Mostra os quatro cenários: alvo na 1ª posição (melhor), no meio (médio), no fim e ausente (pior).
- Mostra que, quando n dobra, o pior caso dobra (crescimento linear).

Perguntas prováveis

P1. Qual é a complexidade da busca linear no pior caso e por quê?

R. $O(n)$. No pior caso (alvo na última posição ou ausente) ela compara o alvo com todos os n elementos.

P2. A busca linear exige lista ordenada?

R. Não. Funciona em qualquer lista; essa é a vantagem dela sobre a busca binária.

P3. Quantos passos no melhor caso? E no caso médio?

R. Melhor: 1 passo (alvo na 1ª posição), $O(1)$. Médio: cerca de $n/2$ passos, que continua sendo $O(n)$ porque a constante $1/2$ é ignorada.

P4. Se a lista dobrar de tamanho, o que acontece com o pior caso?

R. Dobra também: 1.000 elementos $\rightarrow$ 1.000 passos; 2.000 $\rightarrow$ 2.000 passos. É a marca do crescimento linear.

Código

```
def busca_linear(lista, alvo):
    """Devolve (indice ou None, passos)."""
    passos = 0
    for i in range(len(lista)):
        passos += 1                # olhou mais um elemento
        if lista[i] == alvo:
            return i, passos
    return None, passos           # percorreu tudo e nao achou

if __name__ == "__main__":
    n = 1000
    lista = list(range(0, 2 * n, 2))    # 0, 2, 4, ... (n numeros pares, ordenados)
    cenarios = [
        ("melhor caso (1a posicao)", lista[0]),
        ("caso medio (meio)", lista[n // 2]),
        ("pior caso (ultimo)", lista[-1]),
        ("pior caso (ausente)", 1),      # impar: nao esta na lista
    ]
    print(f"BUSCA LINEAR em lista de {n} elementos")
    print("-" * 62)
    for nome, alvo in cenarios:
        idx, passos = busca_linear(lista, alvo)
        print(f"{nome:<27} alvo={alvo:<6} indice={str(idx):<6} passos={passos}")

    print("\nQuando n dobra, o pior caso dobra (O(n)):")
    for n2 in (1000, 2000, 4000, 8000):
```

```
_, p = busca_linear(list(range(n2)), -1)
print(f" n={n2:<6} passos no pior caso={p}")
```

Saída de exemplo (execução real em 20/09/2026)

```
BUSCA LINEAR em lista de 1000 elementos
-----
melhor caso (1a posicao)    alvo=0      indice=0      passos=1
caso medio (meio)         alvo=1000   indice=500    passos=501
pior caso (ultimo)        alvo=1998   indice=999    passos=1000
pior caso (ausente)       alvo=1      indice=None   passos=1000

Quando n dobra, o pior caso dobra (O(n)):
n=1000  passos no pior caso=1000
n=2000  passos no pior caso=2000
n=4000  passos no pior caso=4000
n=8000  passos no pior caso=8000
```

[02] Busca binária iterativa com rastreamento de baixo/alto/meio

Arquivo: scripts/02_busca_binaria.py · **Tema:** Busca · **Complexidade:** melhor $O(1)$, médio e pior $O(\log n)$ · **Memória:** $O(1)$

Palavras-chave: busca binária, lista ordenada, meio, metade, $\log_2$, baixo, alto, chute, descartar metade, passos, 128, 1 milhão

O que o script faz

- Faz a busca binária em uma lista ORDENADA mostrando, a cada passo, o intervalo [baixo, alto], o meio e a decisão tomada.
- Mostra um alvo presente, um no início e um ausente.
- Compara os passos medidos no pior caso com $\text{floor}(\log_2 n) + 1$ e com o teto do livro, $\log_2 n$ arredondado para cima.

Perguntas prováveis

- P1.** Por que a lista precisa estar ordenada?
R. Porque a cada passo o algoritmo descarta uma metade inteira com base na comparação com o elemento do meio; isso só vale se tudo à esquerda for menor e tudo à direita for maior.
- P2.** Quantos passos no máximo para $n = 128$? E para $n = 1.000.000$? (Livro, ex. 1.1)
R. $\log_2 128 = 7$ passos pela conta do livro (uma implementação que conta a comparação final chega a 8). Para 1.000.000: 20 passos, pois $2^{20} = 1.048.576$.
- P3.** O que acontece quando o alvo não está na lista?
R. O intervalo encolhe até baixo ficar maior que alto; o laço termina e devolve None, ainda em $O(\log n)$ passos.
- P4.** Qual é o melhor caso da busca binária?
R. O alvo está exatamente no meio da lista: 1 passo, $O(1)$.
- P5.** Se a lista dobra de tamanho, quantos passos a mais? (Livro, ex. 1.2)
R. Apenas 1 passo a mais: $\log_2(2n) = \log_2 n + 1$.

Código

```
import math

def busca_binaria(lista, alvo, mostrar=False):
    """Devolve (indice ou None, passos). Exige lista ordenada."""
```

```

baixo, alto = 0, len(lista) - 1
passos = 0
while baixo <= alto:
    passos += 1
    meio = (baixo + alto) // 2
    chute = lista[meio]
    if mostrar:
        print(f" passo {passos}: baixo={baixo:<3} alto={alto:<3} meio={meio:<3} chute={chute}")
    if chute == alvo:
        return meio, passos
    if chute < alvo:
        baixo = meio + 1           # alvo so pode estar na metade da direita
    else:
        alto = meio - 1           # alvo so pode estar na metade da esquerda
return None, passos              # intervalo vazio: nao esta na lista

if __name__ == "__main__":
    lista = [3, 8, 12, 19, 25, 31, 37, 42, 49, 56]
    print("Lista ordenada:", lista)
    for alvo in (37, 3, 4):
        print(f"\nProcurando {alvo}:")
        idx, passos = busca_binaria(lista, alvo, mostrar=True)
        print(f" resultado: indice={idx} em {passos} passos")

    print("\nPIOR CASO MEDIDO x TEORIA")
    print(f"{'n':>10} {'pior caso medido':>17} {'floor(log2 n)+1':>16} {'ceil(log2 n) [livro]':>21}")
    for n in (100, 128, 256, 1_000, 240_000, 1_000_000):
        lst = list(range(n))
        pior = max(busca_binaria(lst, alvo)[1] for alvo in (0, n - 1, -1))
        print(f"{'n':>10} {'pior':>17} {'math.floor(math.log2(n)) + 1':>16} {'math.ceil(math.log2(n))':>21}")
    print("\nDobrar n acrescenta apenas 1 passo: crescimento logaritmico, O(log n).")

```

Saída de exemplo (execução real em 20/09/2026)

Lista ordenada: [3, 8, 12, 19, 25, 31, 37, 42, 49, 56]

Procurando 37:

```

passo 1: baixo=0    alto=9    meio=4    chute=25
passo 2: baixo=5    alto=9    meio=7    chute=42
passo 3: baixo=5    alto=6    meio=5    chute=31
passo 4: baixo=6    alto=6    meio=6    chute=37
resultado: indice=6 em 4 passos

```

Procurando 3:

```

passo 1: baixo=0    alto=9    meio=4    chute=25
passo 2: baixo=0    alto=3    meio=1    chute=8
passo 3: baixo=0    alto=0    meio=0    chute=3
resultado: indice=0 em 3 passos

```

Procurando 4:

```

passo 1: baixo=0    alto=9    meio=4    chute=25
passo 2: baixo=0    alto=3    meio=1    chute=8
passo 3: baixo=0    alto=0    meio=0    chute=3
resultado: indice=None em 3 passos

```

PIOR CASO MEDIDO x TEORIA

n	pior caso medido	floor(log2 n)+1	ceil(log2 n) [livro]
100	7	7	7
128	8	8	7
256	9	9	8
1000	10	10	10
240000	18	18	18
1000000	20	20	20

Dobrar n acrescenta apenas 1 passo: crescimento logaritmico, $O(\log n)$.

[03] Busca binária recursiva (dividir para conquistar)

Arquivo: scripts/03_busca_binaria_recursiva.py · **Tema:** Busca / Dividir para conquistar · **Complexidade:** $O(\log n)$ ·

Memória: pilha $O(\log n)$

Palavras-chave: busca binária recursiva, dividir para conquistar, caso base, caso recursivo, metade, profundidade da pilha, livro 4.4

O que o script faz

- Implementa a busca binária como recursão: caso base = intervalo vazio ou alvo no meio; caso recursivo = chamar a função em uma das metades.
- Imprime cada chamada com recuo proporcional à profundidade, para visualizar a pilha.

Perguntas prováveis

P1. Qual é o caso base e o caso recursivo da busca binária? (Livro, ex. 4.4)

R. Caso base: intervalo vazio (não achou) ou elemento do meio igual ao alvo (achou). Caso recursivo: descartar a metade que não pode conter o alvo e repetir a busca na outra metade.

P2. Por que a busca binária é um exemplo de dividir para conquistar?

R. Porque reduz o problema a um subproblema do mesmo tipo com metade do tamanho, até chegar ao caso base.

P3. Qual é a profundidade máxima da pilha nessa versão?

R. Cerca de $\log_2 n$ chamadas empilhadas, uma por metade descartada. Para $n = 10$, no máximo 4.

P4. A versão recursiva é mais rápida que a iterativa?

R. Não; faz os mesmos passos. A recursiva gasta pilha $O(\log n)$ e tem custo de chamada; a iterativa usa memória $O(1)$. A recursiva é mais clara para mostrar o dividir para conquistar.

Código

```
def busca_binaria_rec(lista, alvo, baixo=0, alto=None, profundidade=0):
    if alto is None:
        alto = len(lista) - 1
    recuo = " " * profundidade
    if baixo > alto:
        # caso base 1: intervalo vazio
        print(f"{recuo}[{baixo}..{alto}] vazio -> nao esta na lista")
        return None
    meio = (baixo + alto) // 2
    print(f"{recuo}[{baixo}..{alto}] meio={meio} valor={lista[meio]}")
    if lista[meio] == alvo:
        # caso base 2: achou
        return meio
    if lista[meio] < alvo:
        # caso recursivo: metade da direita
        return busca_binaria_rec(lista, alvo, meio + 1, alto, profundidade + 1)
    return busca_binaria_rec(lista, alvo, baixo, meio - 1, profundidade + 1) # metade da esquerda

if __name__ == "__main__":
    lista = [3, 8, 12, 19, 25, 31, 37, 42, 49, 56]
    for alvo in (37, 4):
        print(f"\nProcurando {alvo} em {lista}")
        r = busca_binaria_rec(lista, alvo)
        print("resultado:", r)
```

Saída de exemplo (execução real em 20/09/2026)

```
Procurando 37 em [3, 8, 12, 19, 25, 31, 37, 42, 49, 56]
[0..9] meio=4 valor=25
  [5..9] meio=7 valor=42
    [5..6] meio=5 valor=31
      [6..6] meio=6 valor=37
resultado: 6
```

```
Procurando 4 em [3, 8, 12, 19, 25, 31, 37, 42, 49, 56]
[0..9] meio=4 valor=25
  [0..3] meio=1 valor=8
    [0..0] meio=0 valor=3
      [1..0] vazio -> nao esta na lista
resultado: None
```

[04] Benchmark: busca linear x busca binária (passos e tempo) — repete a Atividade 01

Arquivo: scripts/04_benchmark_busca.py · **Tema:** Busca / Medição · **Complexidade:** $O(n)$ x $O(\log n)$ · **Memória:** $O(1)$

Palavras-chave: benchmark, tempo, perf_counter, mediana, passos x tempo, busca linear, busca binária, pior caso, 1 milhão, atividade 01, speedup

O que o script faz

- Cria uma lista de 1.000.000 de números e procura o último (pior caso) com os dois algoritmos.
- Conta passos e mede tempo com perf_counter, repetindo 5 vezes e usando a MEDIANA.
- Mostra a razão de passos (50.000x) e a razão de tempo (bem menor), e explica a diferença.

Perguntas prováveis

P1. Por que medir passos E tempo?

R. Passos são propriedade do algoritmo (iguais em qualquer máquina) e provam a classe Big O; tempo é o que o usuário sente e inclui as constantes que o Big O ignora. Quando os dois concordam, a conclusão é robusta.

P2. Por que a razão de tempo é menor que a razão de passos?

R. Passos não são segundos: cada passo da binária faz mais operações (divisão, comparação, dois índices) que o passo da linear, e há custos fixos de chamada e de resolução do relógio.

P3. Por que usar mediana em vez de média?

R. A média é puxada por picos do sistema operacional (outros processos); a mediana ignora esses picos e reflete o custo típico.

P4. Por que usar uma lista grande?

R. Com n pequeno, a diferença entre $O(n)$ e $O(\log n)$ se perde nas constantes e no ruído do relógio.

Código

```
import statistics
import time

def busca_linear(lista, alvo):
    passos = 0
    for i in range(len(lista)):
        passos += 1
        if lista[i] == alvo:
            return i, passos
    return None, passos
```

```

def busca_binaria(lista, alvo):
    baixo, alto, passos = 0, len(lista) - 1, 0
    while baixo <= alto:
        passos += 1
        meio = (baixo + alto) // 2
        if lista[meio] == alvo:
            return meio, passos
        if lista[meio] < alvo:
            baixo = meio + 1
        else:
            alto = meio - 1
    return None, passos

def cronometrar(funcao, lista, alvo, repeticoes=5):
    """Mediana do tempo (ms) de `repeticoes` execucoes."""
    tempos = []
    for _ in range(repeticoes):
        inicio = time.perf_counter()
        funcao(lista, alvo)
        tempos.append((time.perf_counter() - inicio) * 1000)
    return statistics.median(tempos)

if __name__ == "__main__":
    n = 1_000_000
    lista = list(range(n))
    alvo = n - 1  # ultimo elemento = pior caso

    _, passos_lin = busca_linear(lista, alvo)
    _, passos_bin = busca_binaria(lista, alvo)
    t_lin = cronometrar(busca_linear, lista, alvo)
    t_bin = cronometrar(busca_binaria, lista, alvo, repeticoes=50)

    print(f"Lista de {n:_} elementos, alvo = ultimo (pior caso)".replace("_", "."))
    print("-" * 60)
    print(f"Busca linear - O(n)      : {passos_lin:>9} passos | {t_lin:>10.4f} ms")
    print(f"Busca binaria - O(log n) : {passos_bin:>9} passos | {t_bin:>10.4f} ms")
    print(f"\nRazao de passos: {passos_lin / passos_bin:>10.0f} x")
    print(f"Razao de tempo : {t_lin / t_bin:>10.0f} x")
    print("\nA razao de tempo e menor porque cada passo da binaria custa mais que um")
    print("passo da linear e ha custos fixos (chamada, relógio). Passos nao sao segundos.")

```

Saída de exemplo (execução real em 20/09/2026)

```

Lista de 1.000.000 elementos, alvo = ultimo (pior caso)
-----
Busca linear - O(n)      : 1000000 passos | 28.2143 ms
Busca binaria - O(log n) : 20 passos | 0.0015 ms

Razao de passos: 50000 x
Razao de tempo : 18810 x

```

A razao de tempo e menor porque cada passo da binaria custa mais que um passo da linear e ha custos fixos (chamada, relógio). Passos nao sao segundos.

[05] Selection sort com rastreamento de cada passagem

Arquivo: scripts/05_selection_sort.py · **Tema:** Ordenação · **Complexidade:** melhor, médio e pior $O(n^2)$ · **Memória:** $O(1)$

Palavras-chave: selection sort, ordenação por seleção, menor elemento, $n(n-1)/2$, comparações, trocas, sempre quadrático, livro cap. 2

O que o script faz

- Ordena mostrando, a cada passagem, qual foi o menor encontrado e a lista resultante.
- Conta comparações e trocas nos três cenários (ordenada, aleatória, invertida) com $n = 5$.
- Mostra que as comparações são SEMPRE $n(n-1)/2$, independentemente da ordem inicial.

Perguntas prováveis

P1. Como funciona o selection sort?

R. A cada passagem percorre o trecho ainda não ordenado, encontra o MENOR elemento e o coloca na primeira posição livre. Repete $n-1$ vezes.

P2. Por que ele é $O(n^2)$ mesmo no melhor caso?

R. Porque não aproveita a ordem existente: sempre varre todo o trecho restante para achar o menor. Faz $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ comparações em qualquer entrada.

P3. Quantas trocas ele faz no máximo?

R. $n-1$ (uma por passagem, e só quando o menor não está no lugar). Por isso é bom quando a troca é cara.

P4. Por que $O(n^2)$ e não $O(n^2/2)$? (Livro, cap. 2)

R. Big O ignora constantes multiplicativas como o $1/2$; o que importa é o crescimento quadrático.

Código

```
def selection_sort(lista, mostrar=False):
    lista = list(lista)                # copia, nao altera a original
    n = len(lista)
    comparacoes = trocas = 0
    for i in range(n - 1):
        menor = i
        for j in range(i + 1, n):      # procura o menor no trecho restante
            comparacoes += 1
            if lista[j] < lista[menor]:
                menor = j
        if menor != i:
            lista[i], lista[menor] = lista[menor], lista[i]
            trocas += 1
        if mostrar:
            print(f"  passagem {i + 1}: menor={lista[i]} (veio do indice {menor}) -> {lista}")
    return lista, comparacoes, trocas

if __name__ == "__main__":
    cenarios = [
        ("melhor caso (ja ordenada)", [1, 2, 3, 4, 5]),
        ("caso medio (aleatoria)", [3, 5, 1, 4, 2]),
        ("pior caso (invertida)", [5, 4, 3, 2, 1]),
        ("exemplo da revisao", [5, 2, 9, 1, 7]),
    ]
    for nome, lst in cenarios:
        print(f"\n{name}: {lst}")
        ordenada, comp, troc = selection_sort(lst, mostrar=True)
        print(f"  => {ordenada} | comparacoes={comp} trocas={troc}")

    print("\nComparacoes sao SEMPRE n(n-1)/2:")
    for n in (5, 10, 100, 500):
        _, comp, _ = selection_sort(list(range(n, 0, -1)))
        print(f"  n={n:<4} comparacoes={comp:<7} n(n-1)/2={n * (n - 1) // 2}")
```

Saída de exemplo (execução real em 20/09/2026)

```

melhor caso (já ordenada): [1, 2, 3, 4, 5]
passagem 1: menor=1 (veio do índice 0) -> [1, 2, 3, 4, 5]
passagem 2: menor=2 (veio do índice 1) -> [1, 2, 3, 4, 5]
passagem 3: menor=3 (veio do índice 2) -> [1, 2, 3, 4, 5]
passagem 4: menor=4 (veio do índice 3) -> [1, 2, 3, 4, 5]
=> [1, 2, 3, 4, 5] | comparacoes=10 trocas=0

caso medio (aleatoria): [3, 5, 1, 4, 2]
passagem 1: menor=1 (veio do índice 2) -> [1, 5, 3, 4, 2]
passagem 2: menor=2 (veio do índice 4) -> [1, 2, 3, 4, 5]
passagem 3: menor=3 (veio do índice 2) -> [1, 2, 3, 4, 5]
passagem 4: menor=4 (veio do índice 3) -> [1, 2, 3, 4, 5]
=> [1, 2, 3, 4, 5] | comparacoes=10 trocas=2

pior caso (invertida): [5, 4, 3, 2, 1]
passagem 1: menor=1 (veio do índice 4) -> [1, 4, 3, 2, 5]
passagem 2: menor=2 (veio do índice 3) -> [1, 2, 3, 4, 5]
passagem 3: menor=3 (veio do índice 2) -> [1, 2, 3, 4, 5]
passagem 4: menor=4 (veio do índice 3) -> [1, 2, 3, 4, 5]
=> [1, 2, 3, 4, 5] | comparacoes=10 trocas=2

exemplo da revisao: [5, 2, 9, 1, 7]
passagem 1: menor=1 (veio do índice 3) -> [1, 2, 9, 5, 7]
passagem 2: menor=2 (veio do índice 1) -> [1, 2, 9, 5, 7]
passagem 3: menor=5 (veio do índice 3) -> [1, 2, 5, 9, 7]
passagem 4: menor=7 (veio do índice 4) -> [1, 2, 5, 7, 9]
=> [1, 2, 5, 7, 9] | comparacoes=10 trocas=3

Comparacoes sao SEMPRE  $n(n-1)/2$ :
n=5    comparacoes=10     $n(n-1)/2=10$ 
n=10   comparacoes=45     $n(n-1)/2=45$ 
n=100  comparacoes=4950   $n(n-1)/2=4950$ 
n=500  comparacoes=124750  $n(n-1)/2=124750$ 

```

[06] Bubble sort com parada antecipada e rastreamento de cada passagem

Arquivo: scripts/06_bubble_sort.py · **Tema:** Ordenação · **Complexidade:** melhor $O(n)$, médio e pior $O(n^2)$ · **Memória:** $O(1)$

Palavras-chave: bubble sort, ordenação por bolha, vizinhos, troca, passagem, houve_troca, parada antecipada, melhor caso $O(n)$, $n(n-1)/2$, atividade 02

O que o script faz

- Ordena comparando pares VIZINHOS e trocando quando estão fora de ordem; mostra a lista após cada passagem.
- Para quando uma passagem inteira não troca nada (lista já ordenada).
- Conta comparações, trocas e passagens nos três cenários da Atividade 02 ($n = 5$).

Perguntas prováveis

P1. Como funciona o bubble sort?

R. Compara cada par de vizinhos e troca se estiverem fora de ordem; a cada passagem o maior elemento restante “sobe” para o fim. Repete até uma passagem sem trocas.

P2. Por que o melhor caso é $O(n)$?

R. Em lista já ordenada, a 1ª passagem faz $n-1$ comparações e nenhuma troca; o algoritmo percebe e para. Ex.: $n = 5 \rightarrow 4$ comparações, 0 trocas, 1 passagem.

P3. Quantas comparações e trocas no pior caso (lista invertida)?

R. $n(n-1)/2$ comparações e $n(n-1)/2$ trocas, porque TODO par comparado está fora de ordem. Para $n = 5$: 10 e 10.

P4. Por que o laço interno vai até $n - 1$ - passagem?

R. Porque depois de cada passagem o maior elemento já está no fim; a cauda está ordenada e não precisa ser comparada de novo.

P5. Bubble sort ou selection sort para uma lista quase ordenada?

R. Bubble sort: ele detecta a ordem e para cedo. O selection sort faria $n(n-1)/2$ comparações de qualquer jeito.

Código

```
def bubble_sort(lista, mostrar=False):
    lista = list(lista)  # copia, nao altera a original
    n = len(lista)
    comparacoes = trocas = passagens = 0
    for passagem in range(n - 1):  # no maximo n-1 passagens
        houve_troca = False
        passagens += 1
        for i in range(n - 1 - passagem):  # a cauda ja esta ordenada
            comparacoes += 1
            if lista[i] > lista[i + 1]:
                lista[i], lista[i + 1] = lista[i + 1], lista[i]
                trocas += 1
            houve_troca = True
        if mostrar:
            print(f"  passagem {passagem + 1}: {lista} (trocou: {'sim' if houve_troca else 'nao'})")
        if not houve_troca:  # passagem sem troca -> ja ordenada
            break
    return lista, comparacoes, trocas, passagens

if __name__ == "__main__":
    cenarios = [
        ("melhor caso (ja ordenada)", [1, 2, 3, 4, 5]),
        ("caso medio (aleatoria)", [3, 5, 1, 4, 2]),
        ("pior caso (invertida)", [5, 4, 3, 2, 1]),
        ("exemplo da revisao", [5, 2, 9, 1, 7]),
    ]
    for nome, lst in cenarios:
        print(f"\n{n}{nome}: {lst}")
        ordenada, comp, troc, pas = bubble_sort(lst, mostrar=True)
        print(f"  => {ordenada} | comparacoes={comp} trocas={troc} passagens={pas}")

    print("\nPior caso (invertida): comparacoes = trocas = n(n-1)/2")
    for n in (5, 10, 100, 500):
        _, comp, troc, _ = bubble_sort(list(range(n, 0, -1)))
        print(f"  n={n:<4} comparacoes={comp:<7} trocas={troc:<7} n(n-1)/2={n * (n - 1) // 2}")
```

Saída de exemplo (execução real em 20/09/2026)

```
melhor caso (ja ordenada): [1, 2, 3, 4, 5]
```

```

passagem 1: [1, 2, 3, 4, 5] (trocou: nao)
=> [1, 2, 3, 4, 5] | comparacoes=4 trocas=0 passagens=1

caso medio (aleatoria): [3, 5, 1, 4, 2]
passagem 1: [3, 1, 4, 2, 5] (trocou: sim)
passagem 2: [1, 3, 2, 4, 5] (trocou: sim)
passagem 3: [1, 2, 3, 4, 5] (trocou: sim)
passagem 4: [1, 2, 3, 4, 5] (trocou: nao)
=> [1, 2, 3, 4, 5] | comparacoes=10 trocas=6 passagens=4

pior caso (invertida): [5, 4, 3, 2, 1]
passagem 1: [4, 3, 2, 1, 5] (trocou: sim)
passagem 2: [3, 2, 1, 4, 5] (trocou: sim)
passagem 3: [2, 1, 3, 4, 5] (trocou: sim)
passagem 4: [1, 2, 3, 4, 5] (trocou: sim)
=> [1, 2, 3, 4, 5] | comparacoes=10 trocas=10 passagens=4

exemplo da revisao: [5, 2, 9, 1, 7]
passagem 1: [2, 5, 1, 7, 9] (trocou: sim)
passagem 2: [2, 1, 5, 7, 9] (trocou: sim)
passagem 3: [1, 2, 5, 7, 9] (trocou: sim)
passagem 4: [1, 2, 5, 7, 9] (trocou: nao)
=> [1, 2, 5, 7, 9] | comparacoes=10 trocas=5 passagens=4

Pior caso (invertida): comparacoes = trocas = n(n-1)/2
n=5    comparacoes=10    trocas=10    n(n-1)/2=10
n=10   comparacoes=45    trocas=45    n(n-1)/2=45
n=100  comparacoes=4950  trocas=4950  n(n-1)/2=4950
n=500  comparacoes=124750 trocas=124750 n(n-1)/2=124750

```

[07] Crescimento das operações: o que acontece quando n dobra (bubble x selection)

Arquivo: scripts/07_crescimento_ordenacao.py · **Tema:** Ordenação / Big O · **Complexidade:** $O(n^2)$ · **Memória:** $O(1)$

Palavras-chave: n dobra, quadruplica, fator 4, $n(n-1)/2$, comparações, trocas, crescimento, tabela, random.seed, atividade 02, quadrático

O que o script faz

- Roda bubble sort e selection sort para $n = 25, 50, 100, 200, 400$ nos três cenários (ordenada, aleatória, invertida).
- Imprime a tabela de comparações e trocas e a razão entre n e $2n$, mostrando o fator ~ 4 do crescimento quadrático.
- Usa random.seed(42) para o cenário aleatório ser reprodutível.

Perguntas prováveis

- P1.** Quando n dobra, quantas vezes crescem as comparações de um algoritmo $O(n^2)$?
R. Cerca de 4 vezes: $(2n)^2 = 4n^2$. Na tabela, 1.225 -> 4.950 -> 19.900 (fator ~ 4 a cada dobra).
- P2.** Por que o melhor caso do bubble sort cresce só 2x quando n dobra?
R. Porque em lista ordenada ele faz $n-1$ comparações: crescimento linear, $O(n)$.
- P3.** Por que o selection sort faz o mesmo número de comparações nos três cenários?
R. Ele sempre varre o trecho restante inteiro para achar o menor, sem olhar se a lista já está ordenada.
- P4.** Para que serve random.seed(42) aqui?
R. Para o cenário aleatório sortear sempre a mesma lista; sem isso cada execução mediria um teste diferente.

Código

```
import random
```

```

def bubble_sort(lista):
    lista = list(lista); n = len(lista); comp = troc = 0
    for passagem in range(n - 1):
        houve_troca = False
        for i in range(n - 1 - passagem):
            comp += 1
            if lista[i] > lista[i + 1]:
                lista[i], lista[i + 1] = lista[i + 1], lista[i]
                troc += 1; houve_troca = True
        if not houve_troca:
            break
    return comp, troc

def selection_sort(lista):
    lista = list(lista); n = len(lista); comp = troc = 0
    for i in range(n - 1):
        menor = i
        for j in range(i + 1, n):
            comp += 1
            if lista[j] < lista[menor]:
                menor = j
        if menor != i:
            lista[i], lista[menor] = lista[menor], lista[i]; troc += 1
    return comp, troc

if __name__ == "__main__":
    random.seed(42)
    tamanhos = (25, 50, 100, 200, 400)
    print(f"{'n':>5} {'cenario':<10} {'bubble comp':>12} {'bubble troc':>12} {'select comp':>12} {'select troc':>12}")
    print("-" * 68)
    pior_bubble = {}
    for n in tamanhos:
        listas = {
            "melhor": list(range(n)),
            "medio": random.sample(range(n), n),
            "pior": list(range(n, 0, -1)),
        }
        for cenario, lst in listas.items():
            bc, bt = bubble_sort(lst)
            sc, st = selection_sort(lst)
            print(f"{'n':>5} {'cenario':<10} {'bc':>12} {'bt':>12} {'sc':>12} {'st':>12}")
            if cenario == "pior":
                pior_bubble[n] = bc
        print()

    print("Razao entre n e 2n (pior caso, comparacoes do bubble sort):")
    for n in tamanhos[:-1]:
        print(f"{'n':>4} -> {2 * n:<4}: {pior_bubble[2 * n] / pior_bubble[n]:.2f}x (n(n-1)/2: {n*(n-1)//2} -> {2*n*(2*n-1)//2})")
    print("Fator ~4 a cada dobra = crescimento quadratico, O(n^2).")

```

Saída de exemplo (execução real em 20/09/2026)

n	cenario	bubble comp	bubble troc	select comp	select troc
25	melhor	24	0	300	0
25	medio	245	136	300	22
25	pior	300	300	300	12
50	melhor	49	0	1225	0
50	medio	1170	540	1225	46

50 pior	1225	1225	1225	25
100 melhor	99	0	4950	0
100 medio	4950	2814	4950	96
100 pior	4950	4950	4950	50
200 melhor	199	0	19900	0
200 medio	19465	10005	19900	195
200 pior	19900	19900	19900	100
400 melhor	399	0	79800	0
400 medio	79547	39604	79800	394
400 pior	79800	79800	79800	200

Razao entre n e $2n$ (pior caso, comparacoes do bubble sort):

25 -> 50 : 4.08x ($n(n-1)/2$: 300 -> 1225)

50 -> 100 : 4.04x ($n(n-1)/2$: 1225 -> 4950)

100 -> 200 : 4.02x ($n(n-1)/2$: 4950 -> 19900)

200 -> 400 : 4.01x ($n(n-1)/2$: 19900 -> 79800)

Fator ~4 a cada dobra = crescimento quadratico, $O(n^2)$.

[08] Quicksort (versão do livro/aula) com rastreamento de pivô, partição e pilha

Arquivo: scripts/08_quicksort.py · **Tema:** Dividir para conquistar / Ordenação · **Complexidade:** melhor e médio $O(n \log n)$, pior $O(n^2)$ · **Memória:** pilha $O(\log n)$ a $O(n)$ + listas novas $O(n)$

Palavras-chave: quicksort, pivô, particionamento, menores, maiores, caso base, recursão, altura da pilha, chamadas, dividir para conquistar, aula 06

O que o script faz

- Implementa o quicksort do livro (pivô = primeiro elemento, listas novas de menores e maiores).
- Mostra cada chamada com recuo pela profundidade: pivô, menores, maiores e o resultado combinado.
- Conta o total de chamadas e a altura máxima da pilha para uma lista aleatória e para uma já ordenada (pior caso).

Perguntas prováveis

P1. Qual é o caso base do quicksort?

R. Lista com 0 ou 1 elemento: já está ordenada, basta devolvê-la.

P2. Quais são as três etapas do quicksort?

R. 1) escolher o pivô; 2) particionar em menores e maiores que o pivô; 3) chamar o quicksort recursivamente nos dois subarrays e combinar: menores + [pivô] + maiores.

P3. Por que a lista ordenada é o pior caso com pivô no primeiro elemento?

R. Um dos subarrays fica sempre vazio: a lista só diminui de 1 por nível, a pilha tem altura n e cada nível ainda custa $O(n)$, total $O(n^2)$.

P4. O que muda entre melhor e pior caso: o custo de cada nível ou a quantidade de níveis?

R. A quantidade de níveis (altura da pilha): $\log n$ no melhor, n no pior. Cada nível custa $O(n)$ de qualquer jeito.

P5. Quantas chamadas de quicksort ocorrem?

R. Sempre $O(n)$: cada chamada com 2 ou mais elementos gera duas novas. Para [33, 15, 10, 42, 7] são 7 chamadas; para [1, 2, 3, 4, 5] (pior caso) são 9.

Código

```
import sys

sys.setrecursionlimit(5000)
```

```

chamadas = 0
altura_max = 0

def quicksort(array, profundidade=0, mostrar=False):
    global chamadas, altura_max
    chamadas += 1
    altura_max = max(altura_max, profundidade + 1)
    recuo = " " * profundidade
    if len(array) < 2:                                # caso base
        if mostrar:
            print(f"{recuo}quicksort({array}) -> {array} (caso base)")
        return array
    pivo = array[0]
    menores = [x for x in array[1:] if x <= pivo]
    maiores = [x for x in array[1:] if x > pivo]
    if mostrar:
        print(f"{recuo}quicksort({array}): pivo={pivo} menores={menores} maiores={maiores}")
    resultado = quicksort(menores, profundidade + 1, mostrar) + [pivo] + quicksort(maiores,
        profundidade + 1, mostrar)
    if mostrar:
        print(f"{recuo}-> {resultado}")
    return resultado

def medir(array, mostrar=False):
    global chamadas, altura_max
    chamadas = altura_max = 0
    resultado = quicksort(array, mostrar=mostrar)
    return resultado, chamadas, altura_max

if __name__ == "__main__":
    for nome, lst in (("exemplo da aula", [33, 15, 10, 42, 7]), ("pior caso: ja ordenada", [1, 2, 3,
        4, 5])):
        print(f"\n{n}{nome}: {lst}")
        r, c, a = medir(lst, mostrar=True)
        print(f"resultado={r} chamadas={c} altura da pilha={a}")

    print("\nAltura da pilha para n = 1000:")
    import random
    random.seed(42)
    aleatoria = random.sample(range(1000), 1000)
    _, c1, a1 = medir(aleatoria)
    _, c2, a2 = medir(list(range(1000)))
    print(f" lista aleatoria : chamadas={c1:<5} altura={a1:<5} (~log2 n = 10 no ideal)")
    print(f" lista ordenada : chamadas={c2:<5} altura={a2:<5} (= n: pior caso, 0(n^2))")

```

Saída de exemplo (execução real em 20/09/2026)

```

exemplo da aula: [33, 15, 10, 42, 7]
quicksort([33, 15, 10, 42, 7]): pivo=33 menores=[15, 10, 7] maiores=[42]
  quicksort([15, 10, 7]): pivo=15 menores=[10, 7] maiores=[]
    quicksort([10, 7]): pivo=10 menores=[7] maiores=[]
      quicksort([7]) -> [7] (caso base)
      quicksort([]) -> [] (caso base)
    -> [7, 10]
  quicksort([15]) -> [15] (caso base)
-> [7, 10, 15]
quicksort([42]) -> [42] (caso base)
-> [7, 10, 15, 33, 42]
resultado=[7, 10, 15, 33, 42] chamadas=7 altura da pilha=4

```

```

pior caso: ja ordenada: [1, 2, 3, 4, 5]
quicksort([1, 2, 3, 4, 5]): pivo=1 menores=[] maiores=[2, 3, 4, 5]
quicksort([]) -> [] (caso base)
quicksort([2, 3, 4, 5]): pivo=2 menores=[] maiores=[3, 4, 5]
quicksort([]) -> [] (caso base)
quicksort([3, 4, 5]): pivo=3 menores=[] maiores=[4, 5]
quicksort([]) -> [] (caso base)
quicksort([4, 5]): pivo=4 menores=[] maiores=[5]
quicksort([]) -> [] (caso base)
quicksort([5]) -> [5] (caso base)
-> [4, 5]
-> [3, 4, 5]
-> [2, 3, 4, 5]
-> [1, 2, 3, 4, 5]
resultado=[1, 2, 3, 4, 5] chamadas=9 altura da pilha=5

```

Altura da pilha para $n = 1000$:

```

lista aleatoria : chamadas=1337 altura=20 (~log2 n = 10 no ideal)
lista ordenada : chamadas=1999 altura=1000 (= n: pior caso,  $O(n^2)$ )

```

[09] Quicksort: pivô primeiro x meio x aleatório (melhor, médio e pior caso) — experimento da aula 06

Arquivo: scripts/09_quicksort_pivos.py · **Tema:** Dividir para conquistar / Ordenação / Medição · **Complexidade:** $O(n \log n)$ médio, $O(n^2)$ pior · **Memória:** pilha $O(\log n)$ a $O(n)$

Palavras-chave: quicksort, escolha do pivô, pivô aleatório, pivô do meio, random.choice, pior caso, lista ordenada, caso médio, tempo, comparações, altura da pilha, setrecursionlimit

O que o script faz

- Ordena uma lista de 1.000 elementos já ordenada e uma embaralhada com três estratégias de pivô: primeiro, meio e aleatório.
- Para cada combinação mede comparações, altura da pilha e tempo, mostrando o pior caso $O(n^2)$ do pivô fixo em lista ordenada.

Perguntas prováveis

P1. Como evitar o pior caso do quicksort?

R. Não usar pivô fixo no primeiro/último elemento: escolher o do meio, um aleatório (random.choice) ou a mediana de três. Assim a partição tende a ser equilibrada e a altura fica $\sim \log n$.

P2. Por que a aula usou sys.setrecursionlimit?

R. Porque no pior caso a pilha tem altura n (1.000 níveis para $n = 1.000$), acima do limite padrão do Python (~ 1.000), o que causaria RecursionError.

P3. Com pivô aleatório, qual é o Big O esperado?

R. $O(n \log n)$ no caso médio; o pior caso $O(n^2)$ continua possível, mas com probabilidade desprezível.

P4. Merge sort é sempre melhor por garantir $O(n \log n)$?

R. Não necessariamente: o quicksort tem constante menor e costuma ser mais rápido na prática; o merge sort vale quando é preciso garantir o pior caso.

Código

```

import random
import sys
import time

sys.setrecursionlimit(5000)

```

```

comparacoes = 0
altura_max = 0

def quicksort(arr, escolher_pivo, profundidade=0):
    global comparacoes, altura_max
    altura_max = max(altura_max, profundidade + 1)
    if len(arr) <= 1:
        return arr
    i = escolher_pivo(arr)
    pivo = arr[i]
    resto = arr[:i] + arr[i + 1:]
    comparacoes += len(resto) # cada elemento e comparado com o pivo
    menores = [x for x in resto if x <= pivo]
    maiores = [x for x in resto if x > pivo]
    return quicksort(menores, escolher_pivo, profundidade + 1) + [pivo] + quicksort(maiores,
        escolher_pivo, profundidade + 1)

ESTRATEGIAS = {
    "primeiro": lambda arr: 0,
    "meio": lambda arr: len(arr) // 2,
    "aleatorio": lambda arr: random.randrange(len(arr)),
}

def medir(arr, nome):
    global comparacoes, altura_max
    comparacoes = altura_max = 0
    inicio = time.perf_counter()
    r = quicksort(arr, ESTRATEGIAS[nome])
    tempo = (time.perf_counter() - inicio) * 1000
    assert r == sorted(arr) # validar antes de comparar
    return comparacoes, altura_max, tempo

if __name__ == "__main__":
    random.seed(42)
    n = 1000
    ordenada = list(range(n))
    embaralhada = random.sample(range(n), n)
    print(f"n = {n} | n(n-1)/2 = {n*(n-1)//2} | n*log2(n) ~ {n * 10}")
    print(f"{'lista':<12} {'pivo':<10} {'comparacoes':>12} {'altura pilha':>13} {'tempo (ms)':>11}")
    print("-" * 62)
    for nome_lista, lst in (("ordenada", ordenada), ("embaralhada", embaralhada)):
        for pivo in ESTRATEGIAS:
            c, a, t = medir(lst, pivo)
            print(f"{{nome_lista:<12}} {{pivo:<10}} {{c:>12}} {{a:>13}} {{t:>11.3f}}")
        print()
    print("Pivo fixo + lista ordenada: altura = n e comparacoes = n(n-1)/2 -> O(n^2).")
    print("Pivo do meio ou aleatorio: altura ~ log2 n e comparacoes ~ n log n -> O(n log n).")

```

Saída de exemplo (execução real em 20/09/2026)

```

n = 1000 | n(n-1)/2 = 499500 | n*log2(n) ~ 10000
lista      pivo      comparacoes  altura pilha  tempo (ms)
-----
ordenada   primeiro    499500        1000        24.787
ordenada   meio           7987          10          0.598
ordenada   aleatorio      9972          19          0.943

embaralhada primeiro    10429         20          0.909
embaralhada meio         9995          21          0.899

```

embaralhada aleatorio 10536 20 1.087

Pivo fixo + lista ordenada: altura = n e comparacoes = $n(n-1)/2 \rightarrow O(n^2)$.

Pivo do meio ou aleatorio: altura $\sim \log_2 n$ e comparacoes $\sim n \log n \rightarrow O(n \log n)$.

[10] Merge sort com rastreamento das divisões e intercalações

Arquivo: scripts/10_merge_sort.py · **Tema:** Dividir para conquistar / Ordenação · **Complexidade:** melhor, médio e pior $O(n \log n)$ · **Memória:** $O(n)$

Palavras-chave: merge sort, intercalação, merge, dividir ao meio, combinar, $O(n \log n)$ garantido, estável, memória extra, altura $\log n$

O que o script faz

- Divide a lista ao meio recursivamente até listas de 1 elemento e depois intercala (merge) as metades ordenadas.
- Mostra cada merge com recuo pela profundidade e conta comparações e altura da pilha.

Perguntas prováveis

P1. Quais são as etapas dividir, conquistar e combinar no merge sort?

R. Dividir: cortar a lista ao meio. Conquistar: ordenar cada metade recursivamente (caso base: 1 elemento).

Combinar: intercalar as duas metades ordenadas em $O(n)$.

P2. Por que o merge sort é $O(n \log n)$ em todos os casos?

R. Sempre divide ao meio (altura $\log n$) e cada nível intercala n elementos ($O(n)$ por nível), independentemente da ordem inicial.

P3. Qual é a desvantagem do merge sort em relação ao quicksort?

R. Usa $O(n)$ de memória extra para as listas intercaladas e tem constante maior; na prática costuma ser um pouco mais lento.

P4. Quando preferir merge sort?

R. Quando é preciso garantir $O(n \log n)$ no pior caso ou manter a ordem relativa de elementos iguais (estabilidade).

Código

```
comparacoes = 0
altura_max = 0

def merge_sort(array, profundidade=0, mostrar=False):
    global comparacoes, altura_max
    altura_max = max(altura_max, profundidade + 1)
    if len(array) < 2:                                # caso base
        return array
    meio = len(array) // 2
    esquerda = merge_sort(array[:meio], profundidade + 1, mostrar)    # dividir + conquistar
    direita = merge_sort(array[meio:], profundidade + 1, mostrar)
    saida, i, j = [], 0, 0
    while i < len(esquerda) and j < len(direita):      # combinar (merge)
        comparacoes += 1
        if esquerda[i] <= direita[j]:
            saida.append(esquerda[i]); i += 1
        else:
            saida.append(direita[j]); j += 1
    saida += esquerda[i:] + direita[j:]
    if mostrar:
        print(f"{' ' * profundidade}merge {esquerda} + {direita} -> {saida}")
    return saida
```

```

if __name__ == "__main__":
    lista = [38, 27, 43, 3, 9, 82, 10]
    print("Lista:", lista)
    r = merge_sort(lista, mostrar=True)
    print(f"resultado={r} comparacoes={comparacoes} altura da pilha={altura_max}")

    print("\nComparacoes <= n*log2(n) e altura = log2(n)+1 em QUALQUER cenario:")
    import math
    import random
    random.seed(42)
    for n in (8, 64, 1024):
        for nome, lst in (("ordenada", list(range(n))), ("aleatoria", random.sample(range(n), n)),
                          ("invertida", list(range(n, 0, -1)))):
            comparacoes = altura_max = 0
            merge_sort(lst)
            print(f"  n={n:<5} {nome:<10} comparacoes={comparacoes:<6} altura={altura_max:<3}
                  n*log2(n)={int(n * math.log2(n))}")

```

Saída de exemplo (execução real em 20/09/2026)

```

Lista: [38, 27, 43, 3, 9, 82, 10]
  merge [27] + [43] -> [27, 43]
merge [38] + [27, 43] -> [27, 38, 43]
  merge [3] + [9] -> [3, 9]
  merge [82] + [10] -> [10, 82]
merge [3, 9] + [10, 82] -> [3, 9, 10, 82]
merge [27, 38, 43] + [3, 9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82]
resultado=[3, 9, 10, 27, 38, 43, 82] comparacoes=13 altura da pilha=4

```

```

Comparacoes <= n*log2(n) e altura = log2(n)+1 em QUALQUER cenario:
n=8      ordenada  comparacoes=12  altura=4  n*log2(n)=24
n=8      aleatoria comparacoes=14  altura=4  n*log2(n)=24
n=8      invertida comparacoes=12  altura=4  n*log2(n)=24
n=64     ordenada  comparacoes=192  altura=7  n*log2(n)=384
n=64     aleatoria comparacoes=308  altura=7  n*log2(n)=384
n=64     invertida comparacoes=192  altura=7  n*log2(n)=384
n=1024   ordenada  comparacoes=5120 altura=11  n*log2(n)=10240
n=1024   aleatoria comparacoes=8959 altura=11  n*log2(n)=10240
n=1024   invertida comparacoes=5120 altura=11  n*log2(n)=10240

```

[11] Recursão básica: soma, contagem, máximo, fatorial e contagem regressiva (com profundidade da pilha)

Arquivo: scripts/11_recurcao_basica.py · **Tema:** Recursividade · **Complexidade:** O(n) chamadas · **Memória:** pilha O(n)

Palavras-chave: recursão, caso base, caso recursivo, soma recursiva, contar, máximo, fatorial, regressiva, profundidade da pilha, RecursionError, stack overflow, livro 4.1 4.2 4.3

O que o script faz

- Implementa as funções recursivas do livro/aula (soma, contar, máximo, fatorial, contagem regressiva) mostrando cada chamada com recuo.
- Mede a profundidade máxima da pilha.
- Demonstra o que acontece sem caso base: captura o RecursionError.

Perguntas prováveis

P1. O que são caso base e caso recursivo?

R. Caso base: condição em que a função devolve a resposta sem chamar a si mesma (lista vazia, $n \leq 1$). Caso recursivo: a função chama a si mesma com um problema menor, aproximando-se do caso base.

P2. O que acontece se esquecer o caso base? (Livro, ex. 3.2)

R. A função nunca para; a pilha cresce até estourar. Em Python: `RecursionError: maximum recursion depth exceeded` (limite padrão ~ 1.000).

P3. Qual é a profundidade da pilha em `soma([2, 4, 6])`?

R. 4 quadros ao mesmo tempo: `soma([2,4,6])`, `soma([4,6])`, `soma([6])` e `soma([])`. Para n elementos: $n + 1$ chamadas, $O(n)$ tempo e $O(n)$ de pilha.

P4. Em que ordem o fatorial multiplica?

R. De baixo para cima: só depois de chegar ao caso base `fatorial(1) = 1` é que `21`, `32`, `46` e `524 = 120` são calculados nos retornos.

Código

```
import sys

profundidade_atual = 0
profundidade_maxima = 0

def entrar(texto):
    global profundidade_atual, profundidade_maxima
    print(" " * profundidade_atual + texto)
    profundidade_atual += 1
    profundidade_maxima = max(profundidade_maxima, profundidade_atual)

def sair():
    global profundidade_atual
    profundidade_atual -= 1

def soma(lista):
    # Livro ex. 4.1 / aula 06
    entrar(f"soma({lista})")
    if lista == []:
        # caso base
        sair(); return 0
    r = lista[0] + soma(lista[1:])
    # caso recursivo
    sair(); return r

def contar(lista):
    # Livro ex. 4.2
    if lista == []:
        return 0
    return 1 + contar(lista[1:])

def maximo(lista):
    # Livro ex. 4.3
    if len(lista) == 1:
        # caso base: 1 elemento
        return lista[0]
    resto = maximo(lista[1:])
    return lista[0] if lista[0] > resto else resto

def fatorial(n):
    # Livro cap. 3
    entrar(f"fatorial({n})")
    if n <= 1:
        sair(); return 1
```

```

    r = n * fatorial(n - 1)
    print("    " * (profundidade_atual - 1) + f"-> {n} * fatorial({n - 1}) = {r}")
    sair(); return r

def regressiva(i):                                # Livro cap. 3
    print(i, end=" ")
    if i <= 1:                                    # caso base
        print()
        return
    regressiva(i - 1)

def sem_caso_base(n):
    return sem_caso_base(n + 1)                  # nunca para!

if __name__ == "__main__":
    print("soma([2, 4, 6]):")
    print("    resultado =", soma([2, 4, 6]), "| profundidade maxima da pilha =", profundidade_maxima)

    print("\ncontar([7, 8, 9, 10]) =", contar([7, 8, 9, 10]))
    print("maximo([3, 9, 2, 7]) =", maximo([3, 9, 2, 7]))

    profundidade_maxima = 0
    print("\nfatorial(5):")
    print("    resultado =", fatorial(5), "| profundidade maxima da pilha =", profundidade_maxima)

    print("\nregressiva(5):")
    regressiva(5)

    print("\nSem caso base (limite de recursao =", sys.getrecursionlimit(), "):")
    try:
        sem_caso_base(1)
    except RecursionError as erro:
        print("    RecursionError:", erro)

```

Saída de exemplo (execução real em 20/09/2026)

```

soma([2, 4, 6]):
soma([2, 4, 6])
  soma([4, 6])
    soma([6])
      soma([])
        resultado = 12 | profundidade maxima da pilha = 4

contar([7, 8, 9, 10]) = 4
maximo([3, 9, 2, 7]) = 9

fatorial(5):
fatorial(5)
  fatorial(4)
    fatorial(3)
      fatorial(2)
        fatorial(1)
          -> 2 * fatorial(1) = 2
          -> 3 * fatorial(2) = 6
          -> 4 * fatorial(3) = 24
          -> 5 * fatorial(4) = 120
          resultado = 120 | profundidade maxima da pilha = 5

regressiva(5):
5 4 3 2 1

```

Sem caso base (limite de recursao = 1000):
RecursionError: maximum recursion depth exceeded

[12] Pilha de chamadas: o exemplo greet / greet2 do livro (cap. 3)

Arquivo: scripts/12_pilha_de_chamadas.py · **Tema:** Recursividade / Pilha · **Complexidade:** O(1) · **Memória:** pilha proporcional às chamadas aninhadas

Palavras-chave: pilha de chamadas, call stack, push, pop, greet, greet2, bye, chamada incompleta, suspensão, LIFO, livro 3.1

O que o script faz

- Reproduz greet(name), que chama greet2 e bye, imprimindo o estado da pilha a cada entrada e saída de função.
- Mostra que greet fica “incompleta” enquanto greet2 executa, e é retomada depois.

Perguntas prováveis

P1. O que a pilha de chamadas guarda?

R. Um quadro por chamada em andamento, com as variáveis locais dela e o ponto de retorno. A última função chamada fica no topo e é a primeira a sair (LIFO).

P2. Olhando a pilha [greet2 | name: maggie] sobre [greet | name: maggie], o que se conclui? (Livro, ex. 3.1)

R. greet foi chamada primeiro com name = maggie; ela chamou greet2 com o mesmo name; greet está suspensa esperando greet2 terminar; quando greet2 retornar, greet continua de onde parou.

P3. Qual é a relação entre pilha de chamadas e recursão?

R. Cada chamada recursiva empilha um novo quadro; a profundidade da recursão é a altura da pilha. Sem caso base a pilha cresce sem limite (stack overflow).

Código

```
pilha = []

def empilhar(funcao, **variaveis):
    pilha.append((funcao, variaveis))
    print(f"  push {funcao}{variaveis}  ->  pilha: {[f for f, _ in pilha]}")

def desempilhar():
    funcao, _ = pilha.pop()
    print(f"  pop {funcao}  ->  pilha: {[f for f, _ in pilha]}")

def greet2(name):
    empilhar("greet2", name=name)
    print(f"    how are you, {name}?")
    desempilhar()

def bye():
    empilhar("bye")
    print("    ok bye!")
    desempilhar()

def greet(name):
    empilhar("greet", name=name)
    print(f"    hello, {name}!")
    greet2(name)
    # greet fica suspensa aqui ate greet2 voltar
```

```

print("    getting ready to say bye...")
bye()
desempilhar()

if __name__ == "__main__":
    print("greet('maggie'):")
    greet("maggie")
    print("\nA pilha volta a ficar vazia:", pilha)

```

Saída de exemplo (execução real em 20/09/2026)

```

greet('maggie'):
  push greet{'name': 'maggie'} -> pilha: ['greet']
  hello, maggie!
  push greet2{'name': 'maggie'} -> pilha: ['greet', 'greet2']
  how are you, maggie?
  pop greet2 -> pilha: ['greet']
  getting ready to say bye...
  push bye{} -> pilha: ['greet', 'bye']
  ok bye!
  pop bye -> pilha: ['greet']
  pop greet -> pilha: []

A pilha volta a ficar vazia: []

```

[13] Fibonacci recursivo puro: árvore de chamadas, contagem e crescimento exponencial

Arquivo: scripts/13_fibonacci_recursivo.py · **Tema:** Recursividade / Fibonacci · **Complexidade:** tempo $O(2^n)$ (exatamente $\Theta(1,618^n)$) · **Memória:** pilha $O(n)$

Palavras-chave: fibonacci, recursivo puro, árvore de chamadas, subproblemas sobrepostos, recálculo, exponencial, razão áurea, phi 1.618, $2F(n+1)-1$, chamadas, atividade 03

O que o script faz

- Desenha a árvore de chamadas de fib(5) e conta quantas vezes cada subproblema é recalculado.
- Conta as chamadas para $n = 5..30$ e confere a fórmula fechada $T(n) = 2 \cdot F(n+1) - 1$.
- Mostra que a razão $T(n)/T(n-1)$ converge para a razão áurea ($\sim 1,618$): cada +1 em n multiplica o trabalho por 1,618.

Perguntas prováveis

P1. Por que o Fibonacci recursivo puro é ineficiente?

R. Porque recalcula os mesmos subproblemas muitas vezes (fib(2) aparece 3 vezes em fib(5); fib(1), 5 vezes). A árvore de chamadas cresce exponencialmente.

P2. Qual é a complexidade e como se chega a ela?

R. $T(n) = T(n-1) + T(n-2) + 1$, a própria recorrência de Fibonacci; solução $T(n) = 2 \cdot F(n+1) - 1 = \Theta(1,618^n)$, que está em $O(2^n)$. Confirmado: 15, 177, 21.891 e 2.692.537 chamadas para $n = 5, 10, 20, 30$.

P3. Se fib(30) leva 0,13 s, quanto leva fib(40)?

R. Cerca de $0,13 \times 1,618^{10} \approx 0,13 \times 123 \approx 16$ s. Cada +1 em n multiplica o tempo por $\sim 1,618$.

P4. O tempo é exponencial; a memória também?

R. Não. A pilha tem no máximo n quadros ao mesmo tempo (a árvore é percorrida em profundidade): memória $O(n)$.

Código

```

from collections import Counter

chamadas = 0

def fib_recursivo(n):
    global chamadas
    chamadas += 1
    if n < 2:                                     # casos base: F(0) = 0, F(1) = 1
        return n
    return fib_recursivo(n - 1) + fib_recursivo(n - 2)

def contar_chamadas(n):
    global chamadas
    chamadas = 0
    valor = fib_recursivo(n)
    return valor, chamadas

def fib_iterativo(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

def desenhar_arvore(n, prefixo="", ultimo=True, raiz=True, contador=None):
    contador = Counter() if contador is None else contador
    contador[n] += 1
    conector = "" if raiz else ("`- " if ultimo else "|- ")
    print(f"{prefixo}{conector}fib({n}){'  <- caso base' if n < 2 else ''}")
    if n >= 2:
        novo = prefixo if raiz else prefixo + ("  " if ultimo else "| ")
        desenhar_arvore(n - 1, novo, False, False, contador)
        desenhar_arvore(n - 2, novo, True, False, contador)
    return contador

if __name__ == "__main__":
    print("ARVORE DE CHAMADAS DE fib(5)")
    contador = desenhar_arvore(5)
    print("\nQuantas vezes cada subproblema foi calculado:")
    for k in sorted(contador):
        print(f"  fib({k}) -> {contador[k]}x")
    print(f"  total = {sum(contador.values())} chamadas")

    print(f"\n{'n':>3} {'chamadas':>10} {'2F(n+1)-1':>10} {'T(n)/T(n-1)':>12}")
    for n in (5, 10, 15, 20, 21, 22, 23, 24, 25):
        _, c = contar_chamadas(n)
        formula = 2 * fib_iterativo(n + 1) - 1
        anterior = 2 * fib_iterativo(n) - 1          # T(n-1) pela formula fechada
        print(f"{'n':>3} {'c':>10} {'formula':>10} {'c / anterior:>12.4f}")
    print("Razao converge para phi = 1,618 (razao aurea): crescimento exponencial.")

```

Saída de exemplo (execução real em 20/09/2026)

```

ARVORE DE CHAMADAS DE fib(5)
fib(5)
|- fib(4)
|  |- fib(3)
|   |- fib(2)
|    |- fib(1)  <- caso base
|    |- fib(0)  <- caso base

```

```

| | ` - fib(1)  <- caso base
| | ` - fib(2)
| |   | - fib(1)  <- caso base
| |   ` - fib(0)  <- caso base
| ` - fib(3)
|   | - fib(2)
|   |   | - fib(1)  <- caso base
|   |   ` - fib(0)  <- caso base
|   ` - fib(1)  <- caso base

```

Quantas vezes cada subproblema foi calculado:

```

fib(0) -> 3x
fib(1) -> 5x
fib(2) -> 3x
fib(3) -> 2x
fib(4) -> 1x
fib(5) -> 1x
total = 15 chamadas

```

n	chamadas	$2F(n+1)-1$	$T(n)/T(n-1)$
5	15	15	1.6667
10	177	177	1.6239
15	1973	1973	1.6185
20	21891	21891	1.6181
21	35421	35421	1.6181
22	57313	57313	1.6181
23	92735	92735	1.6180
24	150049	150049	1.6180
25	242785	242785	1.6180

Razao converge para $\phi = 1,618$ (razao aurea): crescimento exponencial.

[14] Fibonacci com memorização (cache): dicionário manual e @lru_cache

Arquivo: scripts/14_fibonacci_memo.py · **Tema:** Recursividade / Fibonacci / Cache · **Complexidade:** tempo $O(n)$ ·

Memória: cache $O(n)$ + pilha $O(n)$

Palavras-chave: memorização, memoization, cache, dicionário, lru_cache, cache_info, hits, misses, $2n-1$, top-down, cache quente, RecursionError, atividade 03

O que o script faz

- Implementa fib_memo com um dict e fib_lru com functools.lru_cache, contando as chamadas ($2n - 1$).
- Compara com o recursivo puro para $n = 5..30$ (quantas vezes menos chamadas).
- Mostra cache_info() (acertos e falhas) e o custo de uma consulta com o cache já quente.
- Mostra o RecursionError da versão top-down para n grande e como evitá-lo.

Perguntas prováveis

P1. O que é memorização e por que ela ajuda?

R. Guardar o resultado de cada subproblema na primeira vez que é calculado e reaproveitá-lo depois (consulta $O(1)$ no dicionário). Cada $\text{fib}(k)$ é calculado uma única vez: o total cai de exponencial para linear.

P2. Quantas chamadas faz a versão memoizada?

R. $2n - 1$: cada uma das $n-1$ chaves novas dispara duas chamadas, e as demais caem no cache. Para $n = 10$: 19; $n = 20$: 39; $n = 30$: 59.

P3. Qual é o custo em memória da memorização?

R. $O(n)$ para o cache (uma entrada por valor calculado) mais $O(n)$ de pilha na versão recursiva. É a troca clássica: memória por tempo.

P4. Por que $\text{fib_memo}(5000)$ pode dar `RecursionError` e a iterativa não?

R. A memoizada ainda é recursiva e desce n níveis antes de preencher o cache; a iterativa usa um laço e duas variáveis (pilha constante).

P5. O que é um cache quente?

R. Cache já preenchido por chamadas anteriores; a próxima consulta custa $O(1)$. Em benchmarks o cache deve ser zerado, senão mede-se uma consulta e não o algoritmo.

Código

```
import sys
import time
from functools import lru_cache

chamadas = 0

def fib_memo(n, cache=None):
    """Top-down com dicionario. Cache novo a cada chamada externa."""
    global chamadas
    chamadas += 1
    if cache is None:
        cache = {0: 0, 1: 1}
    if n in cache:
        # ja calculado -> consulta O(1)
        return cache[n]
    cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    return cache[n]

@lru_cache(maxsize=None)
def fib_lru(n):
    if n < 2:
        return n
    return fib_lru(n - 1) + fib_lru(n - 2)

def fib_recursivo_chamadas(n):
    """So para comparar: numero de chamadas do recursivo puro = 2F(n+1)-1."""
    a, b = 0, 1
    for _ in range(n + 1):
        a, b = b, a + b
    return 2 * a - 1

if __name__ == "__main__":
    print(f"{'n':>3} {'recursivo puro':>15} {'memoizado':>10} {'2n-1':>6} {'vezes menos':>12}")
    for n in (5, 10, 15, 20, 25, 30):
        chamadas = 0
        fib_memo(n)
```

```

puro = fib_recursoivo_chamadas(n)
print(f"{n:>3} {puro:>15} {chamadas:>10} {2 * n - 1:>6} {puro / chamadas:>11.0f}x")

print("\n@lru_cache: fib_lru(30) =", fib_lru(30))
print("  cache_info:", fib_lru.cache_info(), " <- misses = valores calculados; hits = reaproveitados")

inicio = time.perf_counter()
fib_lru(300)                                # aquece o cache
t_frio = time.perf_counter() - inicio
inicio = time.perf_counter()
fib_lru(300)                                # cache quente
t_quente = time.perf_counter() - inicio
print(f"\nfib_lru(300): 1a chamada {t_frio * 1e6:.1f} us | 2a chamada (cache quente) {t_quente * 1e6:.2f} us")

print("\nTop-down e recursivo: para n grande estoura a pilha (limite =", sys.getrecursionlimit(), ")")
try:
    fib_memo(5000)
except RecursionError:
    print("  fib_memo(5000) -> RecursionError")
print("  solucao: preencher o cache de baixo para cima (bottom-up) ou usar a versao iterativa.")

```

Saída de exemplo (execução real em 20/09/2026)

n	recursivo puro	memoizado	2n-1	vezes menos
5	15	9	9	2x
10	177	19	19	9x
15	1973	29	29	68x
20	21891	39	39	561x
25	242785	49	49	4955x
30	2692537	59	59	45636x

@lru_cache: fib_lru(30) = 832040
 cache_info: CacheInfo(hits=28, misses=31, maxsize=None, currsize=31) <- misses = valores calculados; hits = reaproveitados

fib_lru(300): 1a chamada 323.5 us | 2a chamada (cache quente) 0.20 us

Top-down e recursivo: para n grande estoura a pilha (limite = 1000)
 fib_memo(5000) -> RecursionError
 solucao: preencher o cache de baixo para cima (bottom-up) ou usar a versao iterativa.

[15] Fibonacci: recursivo x memoizado x iterativo — tempo, chamadas e profundidade da pilha

Arquivo: scripts/15_fibonacci_comparacao.py · **Tema:** Recursividade / Fibonacci / Medição · **Complexidade:** $O(2^n)$ x $O(n)$ x $O(n)$ · **Memória:** $O(n)$ x $O(n)$ x $O(1)$

Palavras-chave: fibonacci iterativo, bottom-up, comparação, benchmark, timeit, mínimo, tempo, memória, profundidade da pilha, getsizeof, memória auxiliar, atividade 03

O que o script faz

- Valida que as três implementações dão os mesmos valores (teste de corretude antes do benchmark).
- Mede o tempo das três para $n = 20..30$ com timeit (menor de várias rodadas), zerando o cache a cada execução.
- Mede a profundidade máxima da pilha e a memória auxiliar (tamanho do cache) de cada versão.

Perguntas prováveis

P1. Qual versão é a mais eficiente e por quê?

R. A iterativa (bottom-up): $O(n)$ de tempo e $O(1)$ de memória, sem risco de estourar a pilha. A memoizada também é $O(n)$ de tempo, mas gasta $O(n)$ de memória; a recursiva pura é $O(2^n)$.

P2. Por que validar a corretude antes do benchmark?

R. Comparar a velocidade de algoritmos que dão respostas diferentes não significa nada; um algoritmo rápido e errado não serve.

P3. Por que usar o menor tempo (ou a mediana) e não a média?

R. O mínimo e a mediana são estáveis; a média é distorcida por picos do sistema operacional.

P4. Por que o cache é zerado a cada execução do benchmark?

R. Para medir a construção da tabela inteira, e não um acerto de cache já pronto.

Código

```
import sys
import timeit

profundidade_atual = profundidade_maxima = 0

def fib_recursivo(n):
    global profundidade_atual, profundidade_maxima
    profundidade_atual += 1
    profundidade_maxima = max(profundidade_maxima, profundidade_atual)
    try:
        if n < 2:
            return n
        return fib_recursivo(n - 1) + fib_recursivo(n - 2)
    finally:
        profundidade_atual -= 1

def fib_memo(n, cache=None):
    global profundidade_atual, profundidade_maxima
    if cache is None:
        cache = {0: 0, 1: 1}
    profundidade_atual += 1
    profundidade_maxima = max(profundidade_maxima, profundidade_atual)
    try:
        if n in cache:
            return cache[n]
        cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
        return cache[n]
    finally:
        profundidade_atual -= 1

def fib_iterativo(n):
    anterior, atual = 0, 1
    for _ in range(n):
        anterior, atual = atual, anterior + atual
    return anterior

def menor_tempo(funcao, n, rodadas=3, repeticoes=1):
    return min(timeit.repeat(lambda: funcao(n), repeat=rodadas, number=repeticoes)) / repeticoes

if __name__ == "__main__":
    # 1) corretude
```

```

esperado = [fib_iterativo(i) for i in range(21)]
assert [fib_recursoivo(i) for i in range(21)] == esperado
assert [fib_memo(i) for i in range(21)] == esperado
print("Corretude OK: as tres versoes coincidem para n = 0..20:", esperado[:12], "...")

# 2) tempo
print(f"\n{'n':>3} {'recursivo (s)':>14} {'memoizado (s)':>14} {'iterativo (s)':>14} {'ganho memo':>11}")
for n in (20, 22, 24, 26, 28):
    tr = menor_tempo(fib_recursoivo, n)
    tm = menor_tempo(fib_memo, n, rodadas=5, repeticoes=200)
    ti = menor_tempo(fib_iterativo, n, rodadas=5, repeticoes=200)
    print(f"{'n':>3} {'tr:>14.6f} {'tm:>14.8f} {'ti:>14.8f} {'tr / tm:>10.0f}x")

# 3) memoria: profundidade da pilha e memoria auxiliar
n = 25
profundidade_atual = profundidade_maxima = 0
fib_recursoivo(n)
prof_rec = profundidade_maxima
profundidade_atual = profundidade_maxima = 0
cache = {0: 0, 1: 1}
fib_memo(n, cache)
prof_memo = profundidade_maxima
print(f"\nMemoria (n = {n}):")
print(f" recursivo pilha maxima={prof_rec:<3} quadros | memoria auxiliar: nenhuma (so a pilha)")
print(f" memoizado pilha maxima={prof_memo:<3} quadros | cache com {len(cache)} entradas (~{sys.getsizeof(cache)} bytes so o dict)")
print(f" iterativo pilha maxima=1 quadro | 2 variaveis inteiras: 0(1)")
print("Recursivo puro: tempo exponencial, mas pilha O(n). Memoizado: troca memoria O(n) por tempo O(n).")

```

Saída de exemplo (execução real em 20/09/2026)

Corretude OK: as tres versoes coincidem para n = 0..20: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89] ...

n	recursivo (s)	memoizado (s)	iterativo (s)	ganho memo
20	0.002018	0.00000468	0.00000037	432x
22	0.005418	0.00000532	0.00000040	1019x
24	0.015819	0.00000655	0.00000051	2413x
26	0.038708	0.00000620	0.00000048	6241x
28	0.103097	0.00000700	0.00000055	14730x

Memoria (n = 25):

```

recursivo pilha maxima=25 quadros | memoria auxiliar: nenhuma (so a pilha)
memoizado pilha maxima=25 quadros | cache com 26 entradas (~1168 bytes so o dict)
iterativo pilha maxima=1 quadro | 2 variaveis inteiras: 0(1)
Recursivo puro: tempo exponencial, mas pilha O(n). Memoizado: troca memoria O(n) por tempo O(n).

```

[16] Maior quadrado da fazenda (algoritmo de Euclides / MDC) — exemplo do livro e da aula 06

Arquivo: scripts/16_euclides_maior_quadrado.py · **Tema:** Dividir para conquistar / Recursividade · **Complexidade:** $O(\log(\min(a, b)))$ · **Memória:** pilha $O(\log)$

Palavras-chave: fazenda, 1680 x 640, maior quadrado, Euclides, MDC, resto da divisão, operador módulo, caso base múltiplo, dividir para conquistar, math.gcd, quadrado de 80

O que o script faz

- Implementa maior_quadrado(lado1, lado2) da aula, imprimindo cada redução (lado1, lado2) -> (lado2, lado1 % lado2) até o caso base.

- Resolve a fazenda 1680 x 640 (resposta 80) e outros terrenos; confere com math.gcd e conta as chamadas.

Perguntas prováveis

P1. Qual é o caso base do problema da fazenda?

R. Quando um lado é múltiplo do outro ($\text{lado1} \% \text{lado2} == 0$): o maior quadrado tem o lado menor. Ex.: 160 x 80 -> quadrados de 80.

P2. Qual é o caso recursivo?

R. Preencher o retângulo com o maior quadrado possível e aplicar o mesmo algoritmo à sobra: $\text{maior_quadrado}(\text{lado2}, \text{lado1} \% \text{lado2})$. O maior quadrado que cabe na sobra é o maior que serve para o terreno inteiro.

P3. Mostre as reduções para 1680 x 640.

R. (1680, 640) -> (640, 400) -> (400, 240) -> (240, 160) -> (160, 80) -> caso base. Resposta: 80 x 80 m, em 5 chamadas.

P4. Que algoritmo clássico é esse?

R. O algoritmo de Euclides para o máximo divisor comum: $\text{MDC}(1680, 640) = 80$. Em Python, $\text{math.gcd}(1680, 640)$.

P5. Por que a complexidade é logarítmica?

R. A cada duas chamadas o menor lado pelo menos cai pela metade, então o número de passos cresce como log do menor lado.

Código

```
import math

chamadas = 0

def maior_quadrado(lado1, lado2, mostrar=True):
    global chamadas
    chamadas += 1
    if mostrar:
        print(f" maior_quadrado({lado1}, {lado2})", end="")
    if lado1 == 0 or lado2 == 0:
        if mostrar:
            print(" -> 0")
        return 0
    if lado1 % lado2 == 0:  # caso base: um lado e multiplo do outro
        if mostrar:
            print(f" -> {lado1} e multiplo de {lado2}: CASO BASE, quadrado de {lado2}")
        return lado2
    if mostrar:
        print(f" -> sobra {lado1} % {lado2} = {lado1 % lado2}")
    return maior_quadrado(lado2, lado1 % lado2, mostrar)  # reduz o problema

if __name__ == "__main__":
    terrenos = [(1680, 640), (1200, 450), (1071, 462), (100, 75), (25, 50)]
    for a, b in terrenos:
        chamadas = 0
        print(f"\nTerreno {a} x {b}:")
        r = maior_quadrado(a, b)
        print(f" => quadrados de {r} x {r} | chamadas={chamadas} | math.gcd={math.gcd(a, b)}")
```

Saída de exemplo (execução real em 20/09/2026)

Terreno 1680 x 640:

```

maior_quadrado(1680, 640) -> sobra 1680 % 640 = 400
maior_quadrado(640, 400) -> sobra 640 % 400 = 240
maior_quadrado(400, 240) -> sobra 400 % 240 = 160
maior_quadrado(240, 160) -> sobra 240 % 160 = 80
maior_quadrado(160, 80) -> 160 e multiplo de 80: CASO BASE, quadrado de 80
=> quadrados de 80 x 80 | chamadas=5 | math.gcd=80

```

Terreno 1200 x 450:

```

maior_quadrado(1200, 450) -> sobra 1200 % 450 = 300
maior_quadrado(450, 300) -> sobra 450 % 300 = 150
maior_quadrado(300, 150) -> 300 e multiplo de 150: CASO BASE, quadrado de 150
=> quadrados de 150 x 150 | chamadas=3 | math.gcd=150

```

Terreno 1071 x 462:

```

maior_quadrado(1071, 462) -> sobra 1071 % 462 = 147
maior_quadrado(462, 147) -> sobra 462 % 147 = 21
maior_quadrado(147, 21) -> 147 e multiplo de 21: CASO BASE, quadrado de 21
=> quadrados de 21 x 21 | chamadas=3 | math.gcd=21

```

Terreno 100 x 75:

```

maior_quadrado(100, 75) -> sobra 100 % 75 = 25
maior_quadrado(75, 25) -> 75 e multiplo de 25: CASO BASE, quadrado de 25
=> quadrados de 25 x 25 | chamadas=2 | math.gcd=25

```

Terreno 25 x 50:

```

maior_quadrado(25, 50) -> sobra 25 % 50 = 25
maior_quadrado(50, 25) -> 50 e multiplo de 25: CASO BASE, quadrado de 25
=> quadrados de 25 x 25 | chamadas=2 | math.gcd=25

```

[17] Reconhecendo o Big O de trechos de código: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$

Arquivo: scripts/17_big_o_trechos.py · **Tema:** Big O · **Complexidade:** uma função para cada classe · **Memória:** $O(1)$ (exceto a exponencial, pilha $O(n)$)

Palavras-chave: big o, classes de complexidade, contar operações, laço duplo, dividir por 2, n dobra, fator, constante, logarítmico, linear, linearítmico, quadrático, exponencial, subconjuntos, identificar complexidade

O que o script faz

- Define seis funções, uma por classe de complexidade, cada uma contando suas operações.
- Executa cada uma para n e para $2n$ e imprime o fator de crescimento, que é a “assinatura” da classe.

Perguntas prováveis

P1. Como identificar o Big O de um trecho de código?

R. Conte quantas vezes a operação mais interna executa em função de n : um laço de 0 a $n \rightarrow O(n)$; dois laços aninhados $\rightarrow O(n^2)$; um laço que divide n por 2 a cada volta $\rightarrow O(\log n)$; laço de n voltas com um interno logarítmico $\rightarrow O(n \log n)$; um número fixo de operações $\rightarrow O(1)$.

P2. Qual é o fator de crescimento de cada classe quando n dobra?

R. $O(1)$: 1x; $O(\log n)$: +1 operação; $O(n)$: 2x; $O(n \log n)$: pouco mais que 2x; $O(n^2)$: 4x; $O(2^n)$: elevado ao quadrado.

P3. for i in range(n): for j in range(i+1, n): ... é $O(n^2)$?

R. Sim: são $n(n-1)/2$ pares, e a constante $1/2$ é ignorada. É o padrão do selection sort e do bubble sort.

P4. Um laço que faz $i = i // 2$ até i chegar a 1 é $O(?)$

R. $O(\log n)$: o número de voltas é o número de vezes que n pode ser dividido por 2, como na busca binária.

Código

```

def constante(n):                                # O(1): nao depende de n
    ops = 0
    primeiro = n; ops += 1
    ultimo = n * 2; ops += 1
    return ops

def logaritmico(n):                              # O(Log n): divide por 2 a cada volta
    ops = 0
    i = n
    while i > 1:
        i //= 2; ops += 1
    return ops

def linear(n):                                   # O(n): um Laco
    ops = 0
    for _ in range(n):
        ops += 1
    return ops

def linearitmico(n):                             # O(n Log n): Laco externo n x interno Log n
    ops = 0
    for _ in range(n):
        j = 1
        while j < n:
            j *= 2; ops += 1
    return ops

def quadratico(n):                              # O(n^2): dois lacos aninhados (pares i<j: n(n-1)/2)
    ops = 0
    for i in range(n):
        for j in range(i + 1, n):
            ops += 1
    return ops

def exponencial(n):                             # O(2^n): todos os subconjuntos de n itens
    ops = 0

    def gerar(i, escolhidos):
        nonlocal ops
        ops += 1
        if i == n:
            return
        gerar(i + 1, escolhidos)          # sem o item i
        gerar(i + 1, escolhidos + [i])    # com o item i

    gerar(0, [])
    return ops

if __name__ == "__main__":
    casos = [
        ("O(1)", constante, 1024),
        ("O(log n)", logaritmico, 1024),
        ("O(n)", linear, 1024),
        ("O(n log n)", linearitmico, 1024),
        ("O(n^2)", quadratico, 1024),
        ("O(2^n)", exponencial, 10),
    ]
    print(f"{'classe':<11} {'n':>6} {'ops(n)':>10} {'ops(2n)':>10} {'fator':>8} assinatura")
    print("-" * 70)

```

```

assinaturas = {
    "O(1)": "nao muda", "O(log n)": "+1 operacao", "O(n)": "dobra (2x)",
    "O(n log n)": "pouco mais que 2x", "O(n^2)": "quadruplica (4x)", "O(2^n)": "eleva ao quadrado",
}
for nome, f, n in casos:
    a, b = f(n), f(2 * n)
    print(f"{nome:<11} {n:>6} {a:>10} {b:>10} {b / a:>7.2f}x   {assinaturas[nome]}")

```

Saída de exemplo (execução real em 20/09/2026)

classe	n	ops(n)	ops(2n)	fator	assinatura
O(1)	1024	2	2	1.00x	nao muda
O(log n)	1024	10	11	1.10x	+1 operacao
O(n)	1024	1024	2048	2.00x	dobra (2x)
O(n log n)	1024	10240	22528	2.20x	pouco mais que 2x
O(n^2)	1024	523776	2096128	4.00x	quadruplica (4x)
O(2^n)	10	2047	2097151	1024.50x	eleva ao quadrado

[18] Como medir tempo sem se enganar: perf_counter, timeit, mediana x média, seed e validação

Arquivo: scripts/18_como_medir.py · **Tema:** Medição / Benchmark · **Complexidade:** — · **Memória:** —

Palavras-chave: perf_counter, time.time, timeit, repeat, mediana, média, mínimo, pico, ruído, random.seed, reprodutibilidade, assert, validar corretude, benchmark honesto, statistics

O que o script faz

- Mostra a diferença entre time.time e time.perf_counter (resolução).
- Simula medições com um pico de sistema operacional e compara média, mediana e mínimo.
- Mostra random.seed garantindo os mesmos sorteios e a validação com assert antes de comparar dois algoritmos.

Perguntas prováveis

P1. Por que repetir a medição e usar mediana ou mínimo?

R. Uma medição isolada de microssegundos fica no ruído do relógio; a média é puxada por picos do sistema operacional; mediana e mínimo ignoram esses picos.

P2. Para que serve random.seed(42)?

R. Reprodutibilidade: cada execução sorteia os mesmos alvos e listas; sem isso cada rodada mede um teste diferente.

P3. Por que validar a corretude antes do benchmark?

R. Um algoritmo rápido e errado não serve como base de comparação. Compara-se a saída com uma referência (sorted, index) usando assert.

P4. Qual a diferença entre contar passos e medir tempo?

R. Passos são propriedade do algoritmo e independem da máquina; tempo depende de CPU, linguagem e sistema, mas captura as constantes que o Big O ignora.

Código

```

import random
import statistics
import time
import timeit

```

```

def bubble_sort(lista):
    lista = list(lista); n = len(lista)
    for p in range(n - 1):
        trocou = False
        for i in range(n - 1 - p):
            if lista[i] > lista[i + 1]:
                lista[i], lista[i + 1] = lista[i + 1], lista[i]; trocou = True
        if not trocou:
            break
    return lista

if __name__ == "__main__":
    print("1) Resolucao do relógio")
    print(f"    time.time          : {time.get_clock_info('time').resolution:.1e} s")
    print(f"    time.perf_counter: {time.get_clock_info('perf_counter').resolution:.1e} s  <- use este")

    print("\n2) Media x mediana x minimo com um pico do sistema operacional")
    medicoes = [1.01, 0.99, 1.02, 1.00, 0.98, 1.01, 9.50]          # ms; o 9,50 e um pico
    print(f"    medicoes : {medicoes}")
    print(f"    media    : {statistics.mean(medicoes):.2f} ms (distorcida pelo pico)")
    print(f"    mediana  : {statistics.median(medicoes):.2f} ms (ignora o pico)")
    print(f"    minimo   : {min(medicoes):.2f} ms (custo 'limpo')")

    print("\n3) random.seed garante os mesmos sorteios")
    random.seed(42); a = [random.randint(0, 99) for _ in range(6)]
    random.seed(42); b = [random.randint(0, 99) for _ in range(6)]
    print(f"    seed 42 -> {a}\n    seed 42 -> {b}\n    iguais? {a == b}")

    print("\n4) Validar antes de medir")
    random.seed(42)
    for _ in range(200):
        lst = [random.randint(0, 1000) for _ in range(random.randint(0, 30))]
        assert bubble_sort(lst) == sorted(lst), "bubble_sort divergiu de sorted()!"
    print("    bubble_sort conferido contra sorted() em 200 listas aleatorias: OK")

    print("\n5) timeit.repeat: menor tempo de varias rodadas")
    lst = random.sample(range(300), 300)
    tempos = timeit.repeat(lambda: bubble_sort(lst), repeat=5, number=1)
    print(f"    5 rodadas (s): {[round(t, 4) for t in tempos]}")
    print(f"    minimo={min(tempos):.4f} s mediana={statistics.median(tempos):.4f} s media={statistics.mean(tempos):.4f} s")

```

Saída de exemplo (execução real em 20/09/2026)

```

1) Resolucao do relógio
time.time          : 1.0e-07 s
time.perf_counter: 1.0e-07 s  <- use este

2) Media x mediana x minimo com um pico do sistema operacional
medicoes : [1.01, 0.99, 1.02, 1.0, 0.98, 1.01, 9.5]
media    : 2.22 ms (distorcida pelo pico)
mediana  : 1.01 ms (ignora o pico)
minimo   : 0.98 ms (custo 'limpo')

3) random.seed garante os mesmos sorteios
seed 42 -> [81, 14, 3, 94, 35, 31]
seed 42 -> [81, 14, 3, 94, 35, 31]
iguais? True

4) Validar antes de medir
bubble_sort conferido contra sorted() em 200 listas aleatorias: OK

```

```
5) timeit.repeat: menor tempo de varias rodadas
5 rodadas (s): [0.0021, 0.002, 0.002, 0.0025, 0.0021]
minimo=0.0020 s mediana=0.0021 s media=0.0022 s
```

[19] Calculadora para a prova: passos da busca binária, $n(n-1)/2$, Fibonacci, chamadas, potências de 2

Arquivo: scripts/19_calculadora_prova.py · **Tema:** Ferramenta / Big O · **Complexidade:** — · **Memória:** —

Palavras-chave: calculadora, log2, potências de 2, ceil, floor, $n(n-1)/2$, fibonacci, $2F(n+1)-1$, $2n-1$, fatorial, tabela, estimar tempo, quanto tempo leva

Uso: python 19_calculadora_prova.py 240000

O que o script faz

- Para um n dado (argumento na linha de comando ou os exemplos padrão) imprime: passos da busca binária (livro e implementação), comparações de selection/bubble no pior caso, $n \cdot \log_2 n$, n^2 , 2^n e $n!$.
- Imprime $F(n)$ e o número de chamadas do Fibonacci recursivo puro e memoizado.
- Estima o tempo para um n maior a partir de um tempo medido e da classe de complexidade.

Perguntas prováveis

P1. Quantos passos a busca binária faz para $n = 240.000$? E para 4 bilhões?

R. $\text{ceil}(\log_2 240.000) = 18$; $\text{ceil}(\log_2 4.000.000.000) = 32$.

P2. Quantas comparações faz o selection sort para $n = 1.000$?

R. $n(n-1)/2 = 499.500$.

P3. Um algoritmo $O(n^2)$ leva 2 s para $n = 1.000$; quanto leva para $n = 10.000$?

R. n cresceu 10x, trabalho 100x: cerca de 200 s. Se fosse $O(n \log n)$: cerca de 13x, ~27 s.

P4. Quantas chamadas faz fib_recursivo(25)? E fib_memo(25)?

R. $2 \cdot F(26) - 1 = 242.785$ e $2 \cdot 25 - 1 = 49$.

Código

```
import math
import sys

def fib(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

def estimar(tempo_medido, n_medido, n_novo, classe):
    f = {
        "O(1)": lambda n: 1, "O(log n)": math.log2, "O(n)": lambda n: n,
        "O(n log n)": lambda n: n * math.log2(n), "O(n^2)": lambda n: n ** 2, "O(2^n)": lambda n: 2
        ** n,
    }[classe]
    return tempo_medido * f(n_novo) / f(n_medido)

def br(x):
    return f"{x:,.0f}".replace(",", ".")
```

```
def fmt(x):
    return br(x) if x < 1e12 else f"{x:.2e}"

if __name__ == "__main__":
    ns = [int(a) for a in sys.argv[1:]] or [100, 128, 1_000, 240_000, 1_000_000, 4_000_000_000]
    print(f"{'n':>14} {'bin. livro':>10} {'bin. impl.':>10} {'n(n-1)/2':>22} {'n*log2 n':>14}")
    print("-" * 76)
    for n in ns:
        print(f"{'br(n):>14} {math.ceil(math.log2(n)):>10} {math.floor(math.log2(n)) + 1:>10} {fmt(n * (n - 1) // 2):>22} {fmt(n * math.log2(n)):>14}")
    print("bin. livro = ceil(log2 n) | bin. impl. = floor(log2 n)+1 (conta a comparacao final)")

    print(f"{'\n':>3} {'F(n)':>10} {'chamadas rec. puro':>19} {'chamadas memo':>14} {'n!':>18} {'2^n':>13}")
    for n in (5, 10, 15, 20, 25, 30):
        print(f"{'n:>3} {br(fib(n)):>10} {br(2 * fib(n + 1) - 1):>19} {2 * n - 1:>14} {br(math.factorial(n)) if n <= 15 else '> 10^18':>18} {br(2 ** n):>13}")

    print("\nPotencias de 2:")
    print(" " + " ".join(f"2^{k}={br(2 ** k)}" for k in range(1, 21)))

    print("\nEstimativa: O(n^2) leva 2 s para n = 1.000. Para n = 10.000:")
    for classe in ("O(n)", "O(n log n)", "O(n^2)"):
        print(f" se fosse {classe:<10}: {estimar(2, 1_000, 10_000, classe):>8.1f} s")
```

Saída de exemplo (execução real em 20/09/2026)

n	bin. livro	bin. impl.	n(n-1)/2	n*log2 n
100	7	7	4.950	664
128	7	8	8.128	896
1.000	10	10	499.500	9.966
240.000	18	18	28.799.880.000	4.289.442
1.000.000	20	20	499.999.500.000	19.931.569
4.000.000.000	32	32	8.00e+18	127.589.411.416

bin. livro = ceil(log2 n) | bin. impl. = floor(log2 n)+1 (conta a comparacao final)

n	F(n)	chamadas rec. puro	chamadas memo	n!	2^n
5	5	15	9	120	32
10	55	177	19	3.628.800	1.024
15	610	1.973	29	1.307.674.368.000	32.768
20	6.765	21.891	39	> 10^18	1.048.576
25	75.025	242.785	49	> 10^18	33.554.432
30	832.040	2.692.537	59	> 10^18	1.073.741.824

Potencias de 2:

2^1=2 2^2=4 2^3=8 2^4=16 2^5=32 2^6=64 2^7=128 2^8=256 2^9=512 2^10=1.024 2^11=2.048
2^12=4.096 2^13=8.192 2^14=16.384 2^15=32.768 2^16=65.536 2^17=131.072 2^18=262.144
2^19=524.288 2^20=1.048.576

Estimativa: O(n^2) leva 2 s para n = 1.000. Para n = 10.000:

se fosse O(n) : 20.0 s
se fosse O(n log n): 26.7 s
se fosse O(n^2) : 200.0 s

[20] Caixeiro-viajante por força bruta: O(n!) na prática (livro, cap. 1)

Arquivo: scripts/20_caixeiro_viajante.py · Tema: Big O / Força bruta · Complexidade: O(n!) · Memória: O(n)

Palavras-chave: caixeiro viajante, travelling salesman, permutações, força bruta, fatorial, n!, itertools.permutations, inviável, explode, rotas

O que o script faz

- Gera todas as rotas possíveis entre n cidades (permutações) e escolhe a mais curta.
- Mostra o número de rotas e o tempo para $n = 4..9$, evidenciando o crescimento fatorial.
- Estima quanto tempo levaria para 12, 15 e 20 cidades.

Perguntas prováveis

P1. Por que o caixeiro-viajante por força bruta é $O(n!)$?

R. Porque testa todas as ordens possíveis de visitar as n cidades, e há $n!$ permutações (5 cidades -> 120 rotas; 10 -> 3.628.800).

P2. O que acontece quando se acrescenta uma cidade?

R. O número de rotas é multiplicado por n: de 7 para 8 cidades o trabalho cresce 8 vezes. Por isso é inviável para n grande.

P3. Qual é o algoritmo mais lento da tabela do livro?

R. $O(n!)$. Com $n = 16$ e um computador de 10 operações por segundo, levaria cerca de 66 mil anos.

Código

```
import itertools
import math
import random
import time

def distancia(a, b):
    return math.dist(a, b)

def melhor_rota(cidades):
    origem = cidades[0]
    melhor, menor = None, float("inf")
    rotas = 0
    for perm in itertools.permutations(cidades[1:]):          # (n-1)! rotas saindo da origem
        rotas += 1
        rota = (origem,) + perm + (origem,)
        total = sum(distancia(rota[i], rota[i + 1]) for i in range(len(rota) - 1))
        if total < menor:
            melhor, menor = rota, total
    return melhor, menor, rotas

if __name__ == "__main__":
    random.seed(42)
    print(f"{'cidades':>8} {'rotas (n-1)!':>13} {'tempo (s)':>10} {'crescimento':>12}")
    print("-" * 48)
    anterior = None
    tempos = {}
    for n in range(4, 10):
        cidades = [(random.random() * 100, random.random() * 100) for _ in range(n)]
        inicio = time.perf_counter()
        _, _, rotas = melhor_rota(cidades)
        t = time.perf_counter() - inicio
        tempos[n] = t
        cresc = f"{t / anterior:.1f}x" if anterior else "-"
        print(f"{'n':>8} {'rotas:>13} {'t:>10.4f} {'cresc:>12}")
        anterior = t

    por_rota = tempos[9] / math.factorial(8)
    print("\nEstimativa com o mesmo custo por rota:")
```

```

for n in (12, 15, 20):
    segundos = por_rota * math.factorial(n - 1)
    if segundos < 3600:
        texto = f"{segundos:.0f} s"
    elif segundos < 86400 * 365:
        texto = f"{segundos / 86400:.1f} dias"
    else:
        texto = f"{segundos / (86400 * 365.25):.0f} anos".replace(",", ".")
    print(f" {n} cidades: {math.factorial(n - 1):>22,} rotas -> {texto}".replace(",", "."))

```

Saída de exemplo (execução real em 20/09/2026)

cidades	rotas (n-1)!	tempo (s)	crescimento
4	6	0.0000	-
5	24	0.0000	1.2x
6	120	0.0001	4.6x
7	720	0.0008	6.3x
8	5040	0.0060	7.7x
9	40320	0.0542	9.0x

Estimativa com o mesmo custo por rota:

```

12 cidades:          39.916.800 rotas -> 54 s
15 cidades:          87.178.291.200 rotas -> 1.4 dias
20 cidades: 121.645.100.408.832.000 rotas -> 5.181 anos

```